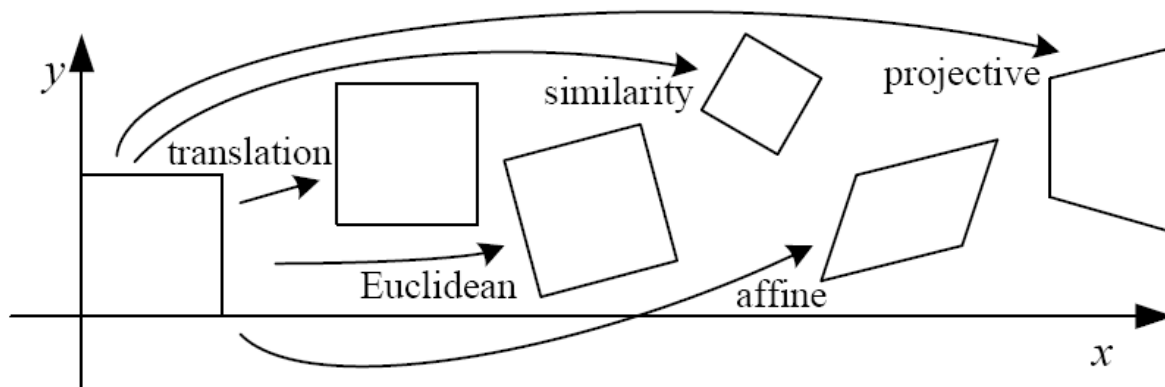


Computer Vision: AI3619

Image Alignment, Transforms, & Panoramas



Slides from Noah Snavely

3 November 2025

Computer Vision, SJTU, Wei Shen

<http://www.wired.com/gadgetlab/2010/07/camera-software-lets-you-see-into-the-past/>

320 GIGAPIXEL Panorama!



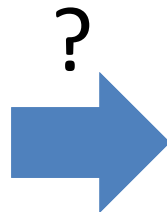
<http://btlondon2012.co.uk/pano.html>

Image alignment



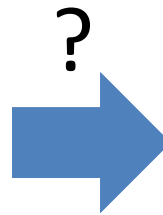
Why don't these images line up exactly?

What is the geometric relationship between these two images?

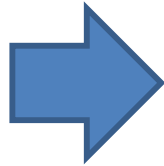


Answer: Similarity transformation (translation, rotation, uniform scale)

What is the geometric relationship between these two images?



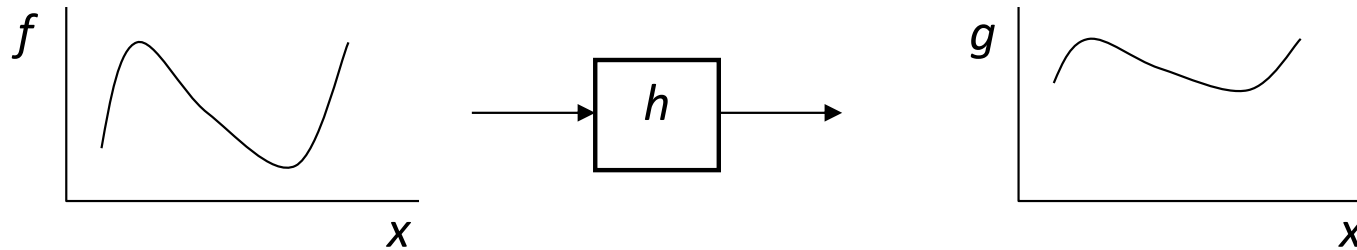
What is the geometric relationship between these two images?



Very important for creating mosaics!

Image Warping

- image filtering: change *range* of image
 - $g(x) = h(f(x))$



- image warping: change *domain* of image
 - $g(x) = f(h(x))$

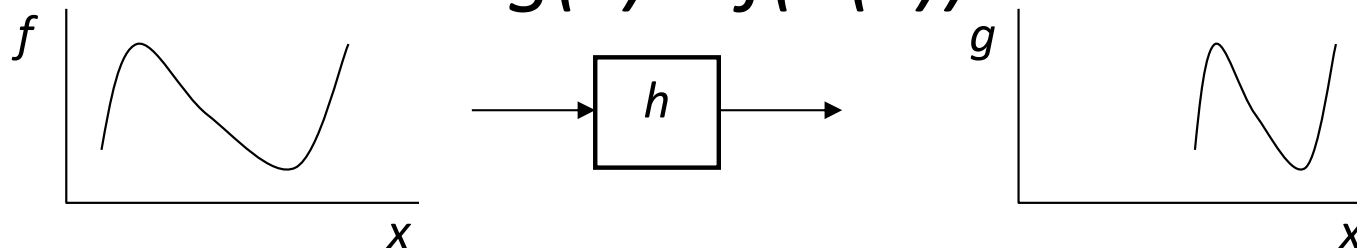
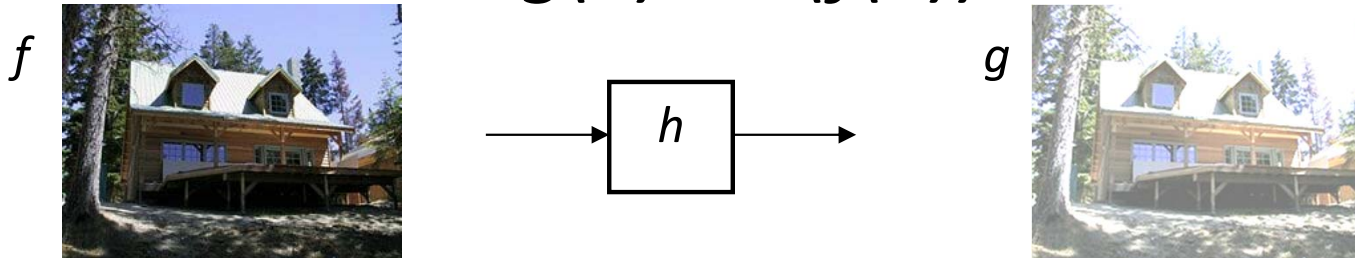


Image Warping

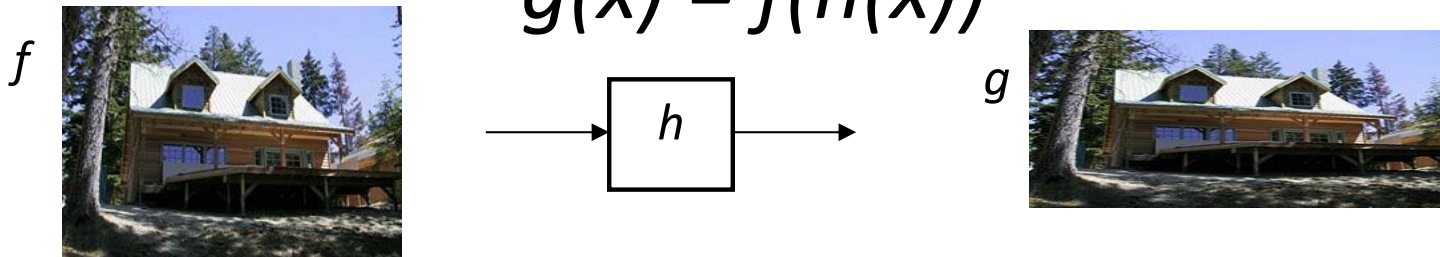
- image filtering: change *range* of image

- $g(x) = h(f(x))$



- image warping: change *domain* of image

- $g(x) = f(h(x))$



Parametric (global) warping

- Examples of parametric warps:



translation



rotation

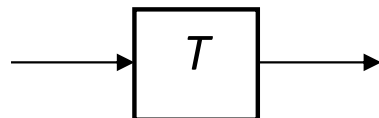


aspect

Parametric (global) warping



$\mathbf{p} = (x, y)$



$\mathbf{p}' = (x', y')$

- Transformation T is a coordinate-changing machine:

$$\mathbf{p}' = T(\mathbf{p})$$

- What does it mean that T is global?
 - Is the same for any point $\mathbf{p}$
 - can be described by just a few numbers (parameters)
- Let's consider *linear* xforms (can be represented by a 2D matrix):

$$\mathbf{p}' = \mathbf{T}\mathbf{p} \quad \begin{bmatrix} x' \\ y' \end{bmatrix} = \mathbf{T} \begin{bmatrix} x \\ y \end{bmatrix}$$

Common linear transformations

- Uniform scaling by s :

$(0,0)$ ●



$(0,0)$ ●

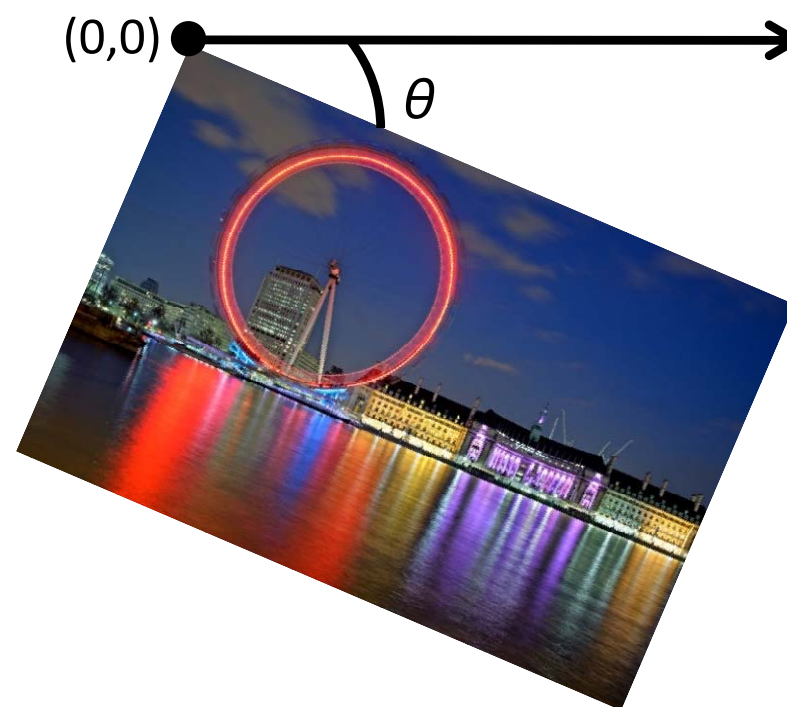


$$\mathbf{S} = \begin{bmatrix} s & 0 \\ 0 & s \end{bmatrix}$$

What is the inverse?

Common linear transformations

- Rotation by angle θ (about the origin)



$$\mathbf{R} = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}$$

What is the inverse?

For rotations:

$$\mathbf{R}^{-1} = \mathbf{R}^T$$

2x2 Matrices

- What types of transformations can be represented with a 2x2 matrix?

2D mirror about Y axis?

$$\begin{aligned}x' &= -x \\ y' &= y\end{aligned}\quad \mathbf{T} = \begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix}$$

2D mirror across line $y = x$?

$$\begin{aligned}x' &= y \\ y' &= x\end{aligned}\quad \mathbf{T} = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

2x2 Matrices

- What types of transformations can be represented with a 2x2 matrix?

2D Translation?

$$\begin{aligned}x' &= x + t_x \\ y' &= y + t_y\end{aligned}\quad \text{NO!}$$

Translation is not a linear operation on 2D coordinates

All 2D Linear Transformations

- Linear transformations are combinations of ...

- Scale,
- Rotation,
- Shear, and
- Mirror

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

- Properties of linear transformations:

- Origin maps to origin
- Lines map to lines
- Parallel lines remain parallel
- Closed under composition

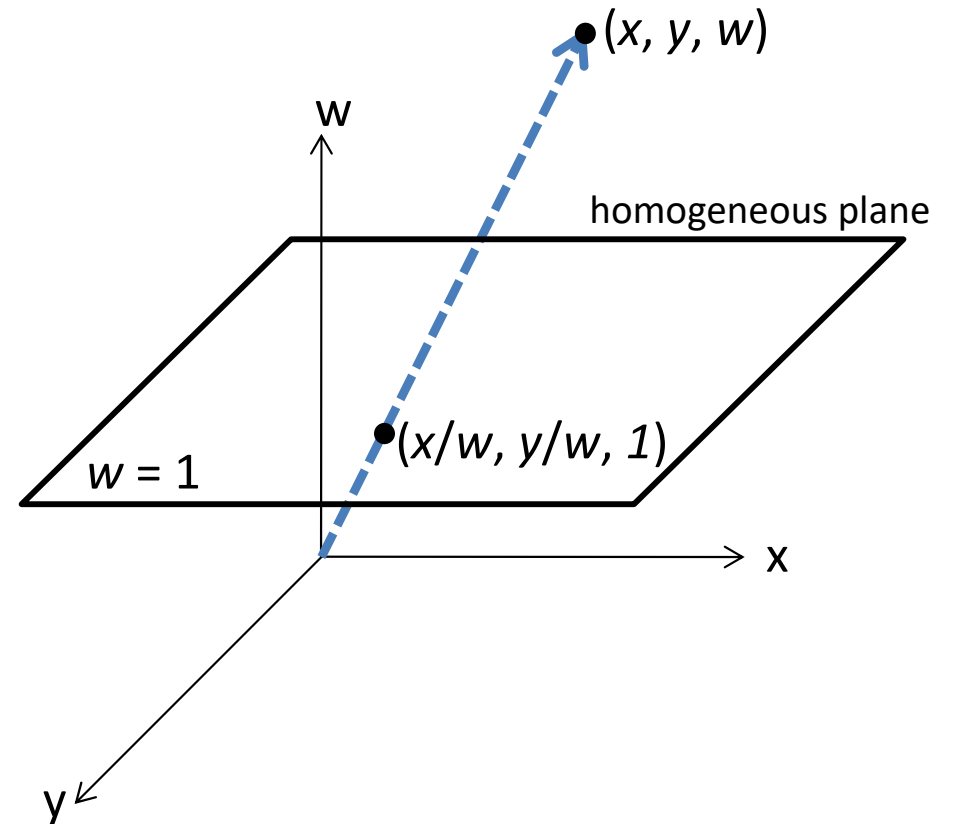
$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} e & f \\ g & h \end{bmatrix} \begin{bmatrix} i & j \\ k & l \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

Homogeneous coordinates

Trick: add one more coordinate:

$$(x, y) \Rightarrow \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

homogeneous image
coordinates



Converting *from* homogeneous coordinates

$$\begin{bmatrix} x \\ y \\ w \end{bmatrix} \Rightarrow (x/w, y/w)$$

Translation

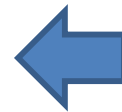
- Solution: homogeneous coordinates to the rescue

$$\mathbf{T} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} x + t_x \\ y + t_y \\ 1 \end{bmatrix}$$

Affine transformations

$$\mathbf{T} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix}$$



any transformation with
last row $[0 \ 0 \ 1]$ we call an
affine transformation

$$\begin{bmatrix} a & b & c \\ d & e & f \\ 0 & 0 & 1 \end{bmatrix}$$

Basic affine transformations

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Translate

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Scale

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

2D *in-plane* rotation

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & sh_x & 0 \\ sh_y & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Shear

Affine Transformations

- Affine transformations are combinations of ...

- Linear transformations, and
- Translations

$$\begin{bmatrix} x' \\ y' \\ w \end{bmatrix} = \begin{bmatrix} a & b & c \\ d & e & f \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ w \end{bmatrix}$$

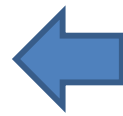
- Properties of affine transformations:
 - Origin does not necessarily map to origin
 - Lines map to lines
 - Parallel lines remain parallel
 - Closed under composition

Is this an affine transformation?



Where do we go from here?

$$\begin{bmatrix} a & b & c \\ d & e & f \\ 0 & 0 & 1 \end{bmatrix}$$



what happens when we
mess with this row?

affine transformation

Projective Transformations aka Homographies aka Planar Perspective Maps

$$\mathbf{H} = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & 1 \end{bmatrix}$$

Called a *homography*
(or *planar perspective map*)

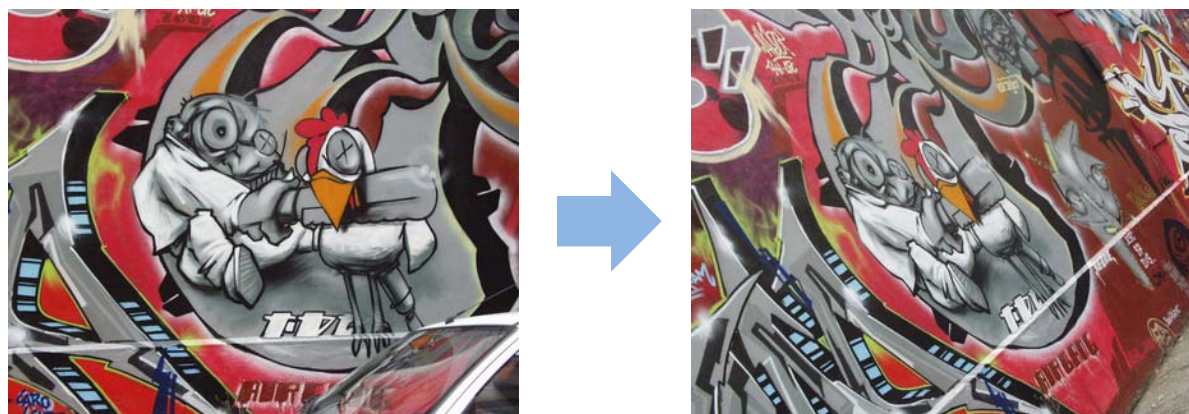
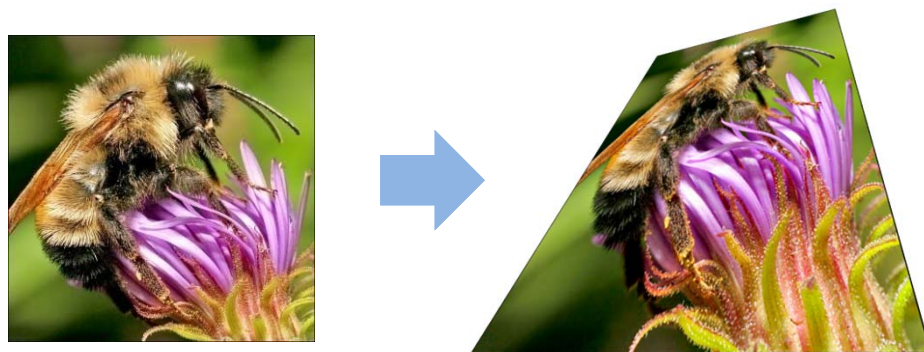
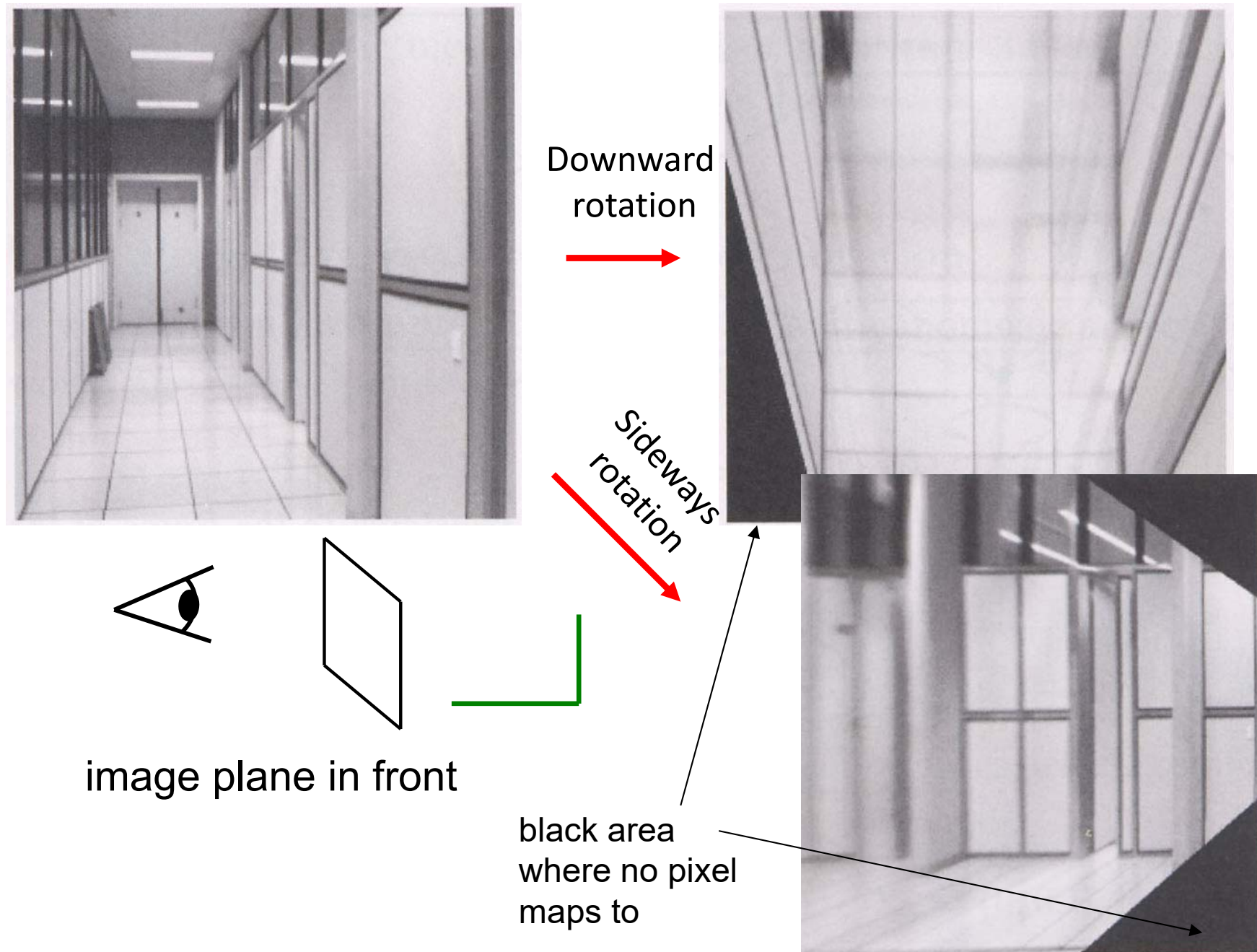
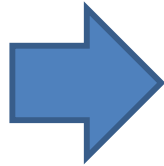


Image warping with homographies



Homographies



Projective Transformations

- Projective transformations ...

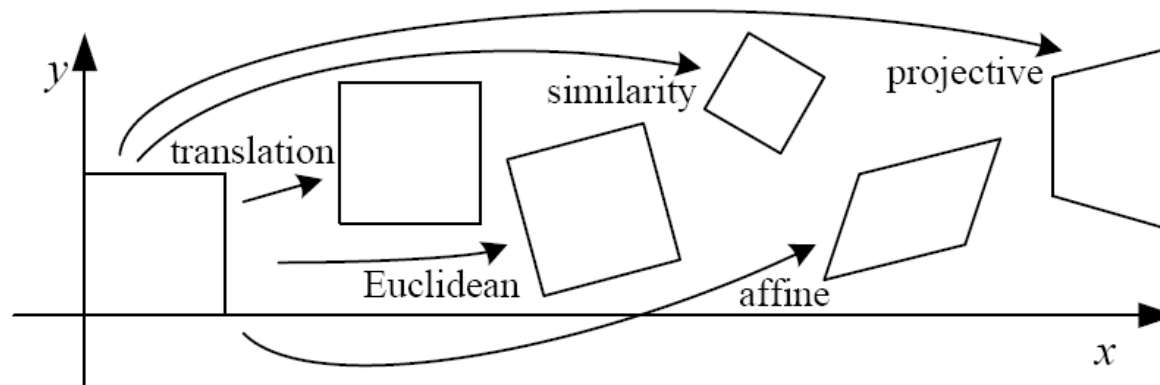
- Affine transformations, and
- Projective warps

$$\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix} \begin{bmatrix} x \\ y \\ w \end{bmatrix}$$

- Properties of projective transformations:

- Origin does not necessarily map to origin
- Lines map to lines
- Parallel lines do not necessarily remain parallel
- Closed under composition

2D image transformations



Name	Matrix	# D.O.F.	Preserves:	Icon
translation	$\begin{bmatrix} I & t \end{bmatrix}_{2 \times 3}$	2	orientation + ...	
rigid (Euclidean)	$\begin{bmatrix} R & t \end{bmatrix}_{2 \times 3}$	3	lengths + ...	
similarity	$\begin{bmatrix} sR & t \end{bmatrix}_{2 \times 3}$	4	angles + ...	
affine	$\begin{bmatrix} A \end{bmatrix}_{2 \times 3}$	6	parallelism + ...	
projective	$\begin{bmatrix} \tilde{H} \end{bmatrix}_{3 \times 3}$	8	straight lines	

These transformations are a nested set of groups

- Closed under composition and inverse is a member

Homographies

$$\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

What happens when
the denominator is 0?

$$\sim \begin{bmatrix} \frac{ax+by+c}{gx+hy+1} \\ \frac{dx+ey+f}{gx+hy+1} \\ 1 \end{bmatrix}$$

Points at infinity

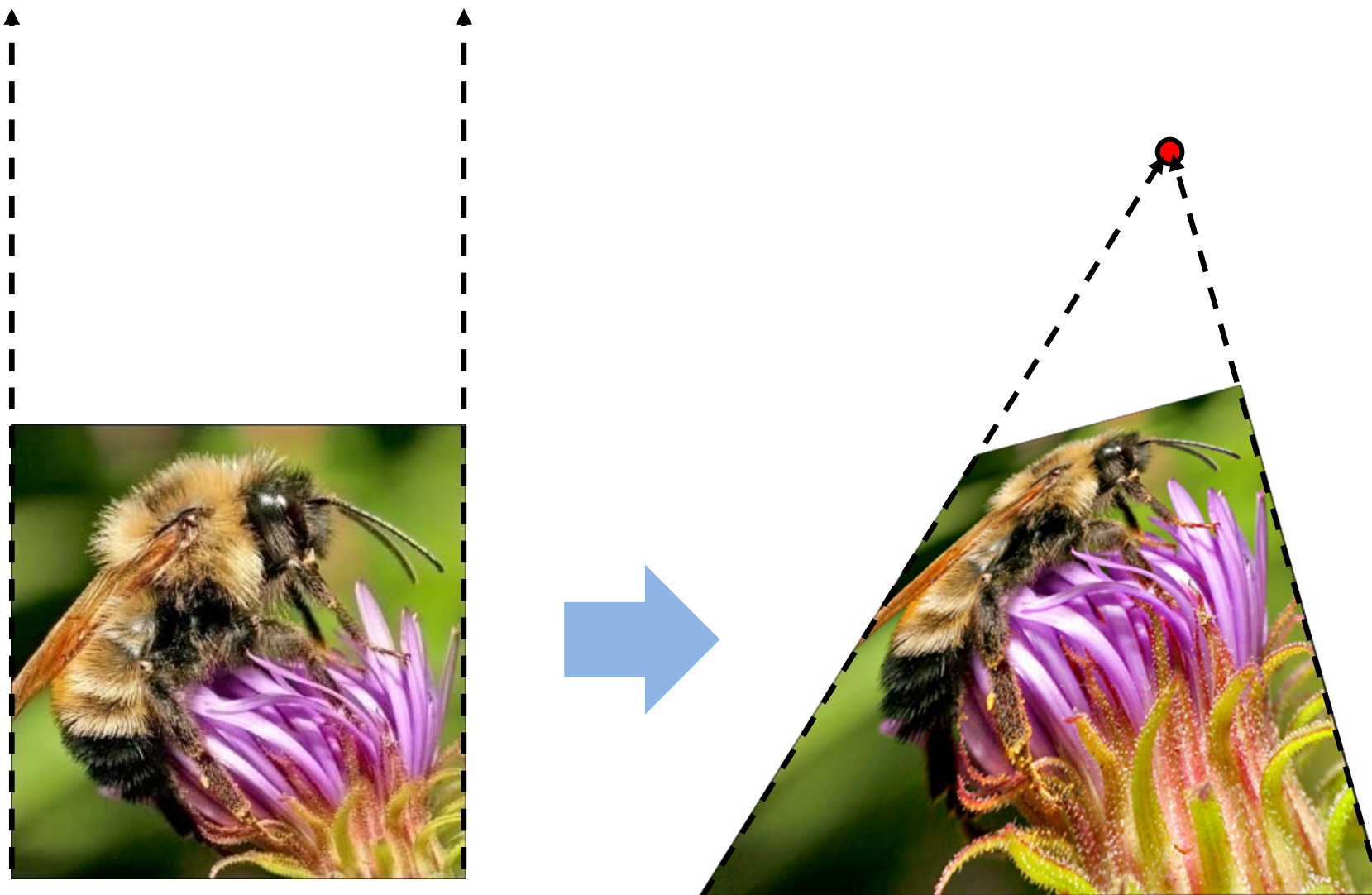


Image warping with homographies

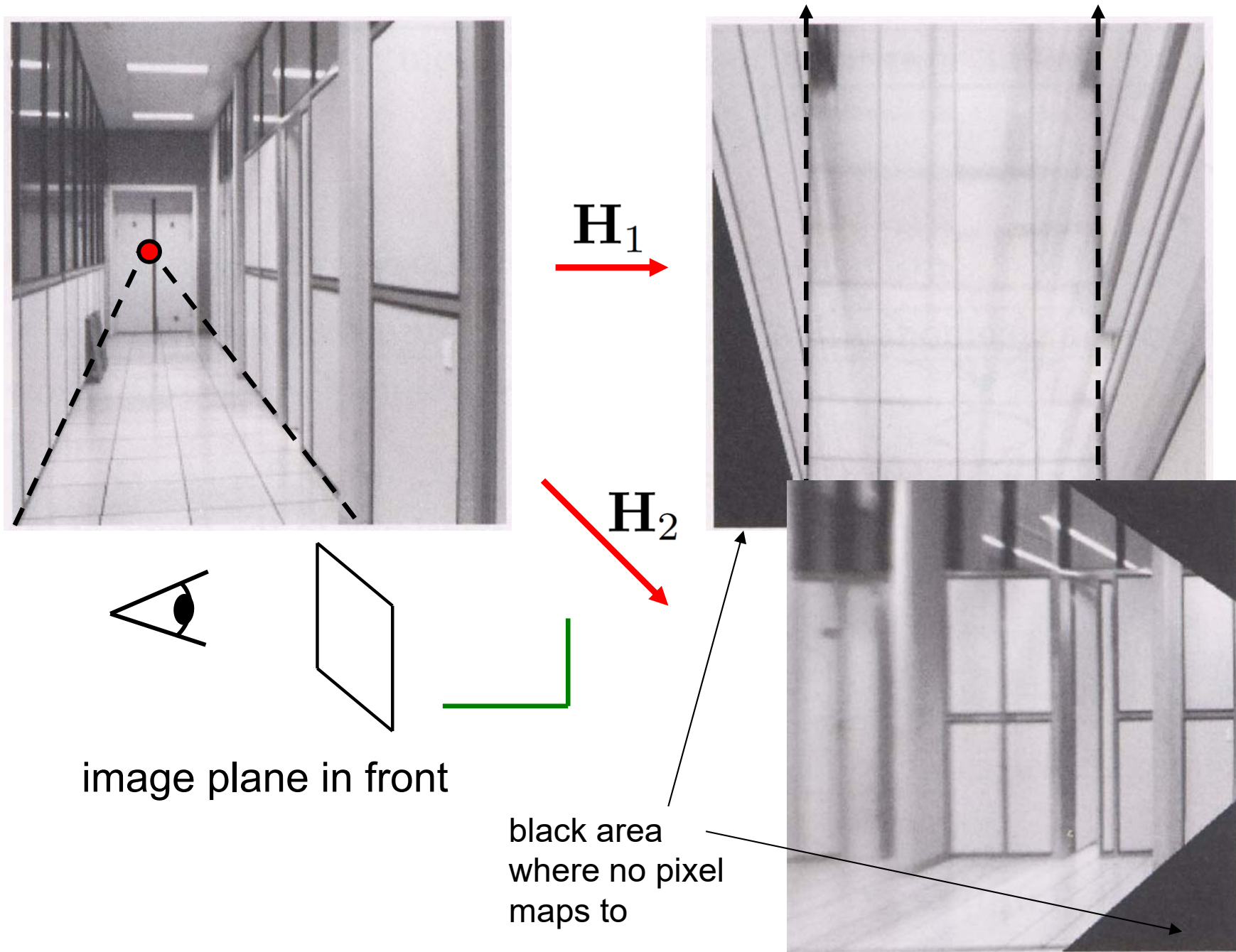
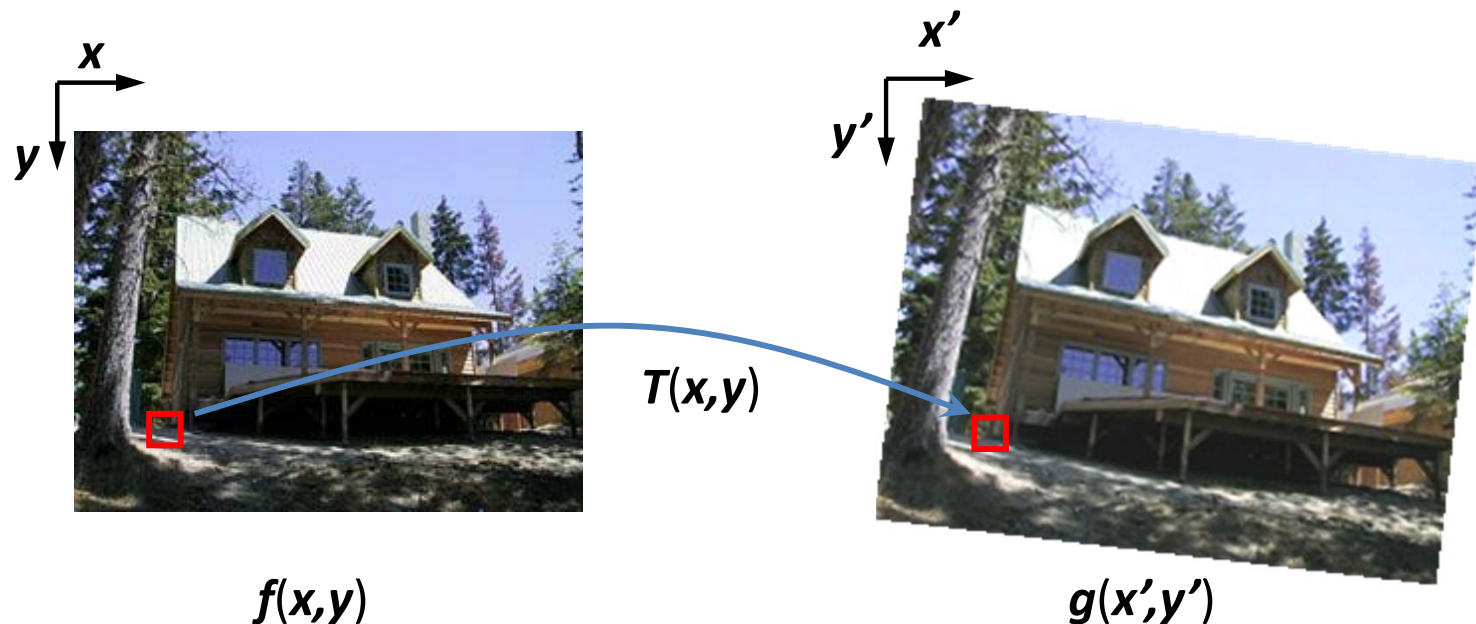


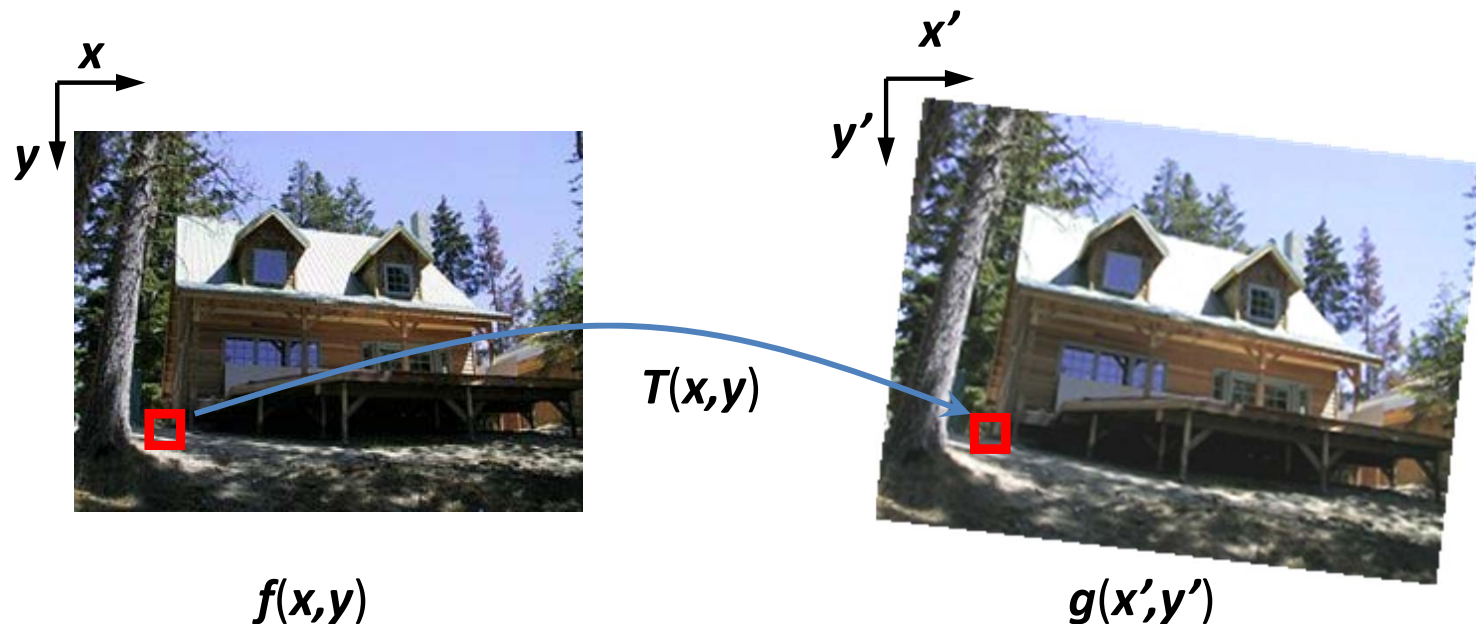
Image Warping

- Given a coordinate xform $(x', y') = T(x, y)$ and a source image $f(x, y)$, how do we compute an xformed image $g(x', y') = f(T(x, y))$?



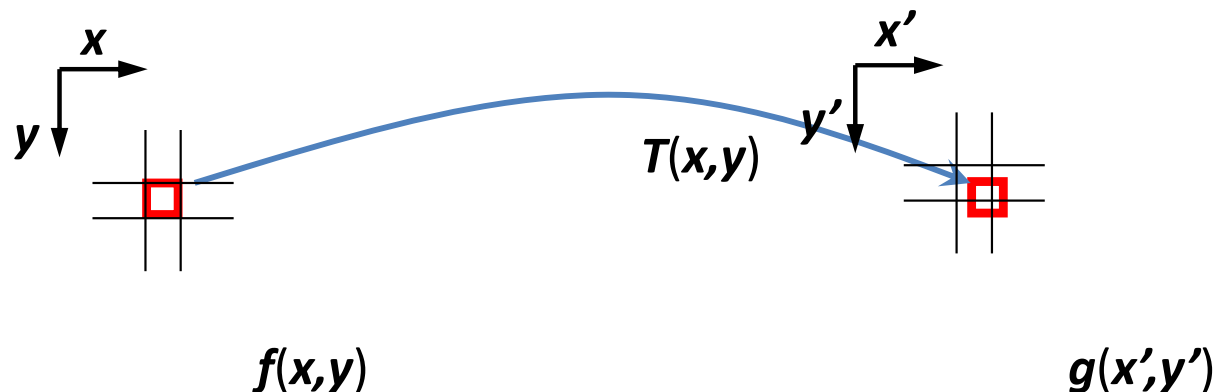
Forward Warping

- Send each pixel $f(\mathbf{x})$ to its corresponding location $(\mathbf{x}', \mathbf{y}') = T(\mathbf{x}, \mathbf{y})$ in $g(\mathbf{x}', \mathbf{y}')$
- What if pixel lands “between” two pixels?



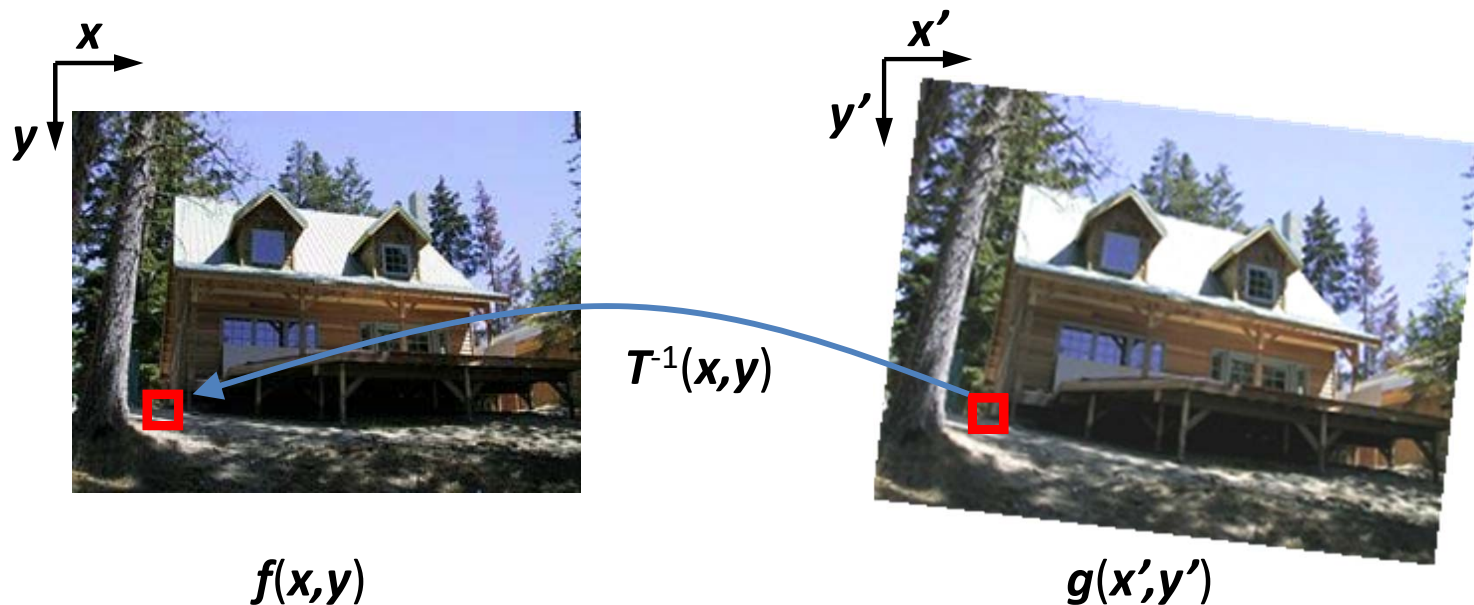
Forward Warping

- Send each pixel $f(x,y)$ to its corresponding location $x' = h(x,y)$ in $g(x',y')$
- What if pixel lands “between” two pixels?
- Answer: add “contribution” to several pixels, normalize later (*splatting*)
- Can still result in holes



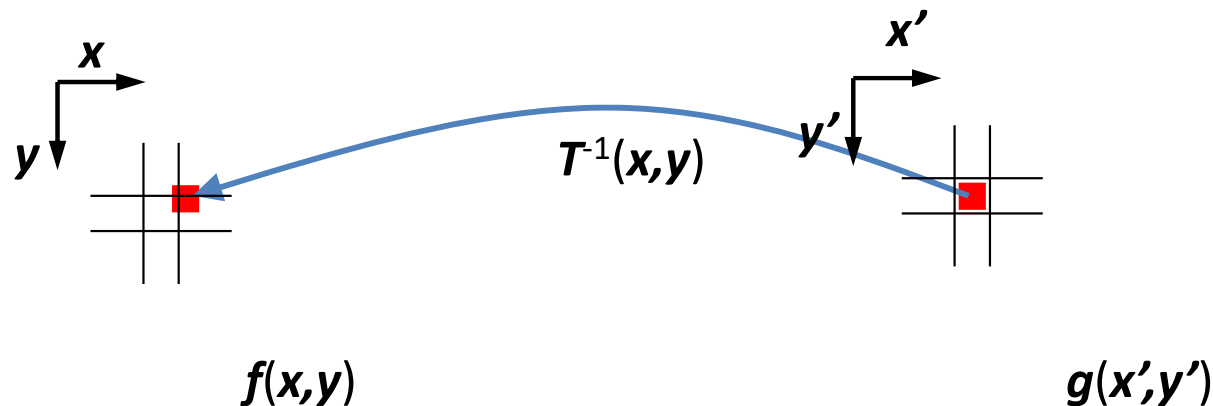
Inverse Warping

- Get each pixel $g(x',y')$ from its corresponding location $(x,y) = T^{-1}(x',y')$ in $f(x,y)$
- Requires taking the inverse of the transform
- What if pixel comes from “between” two pixels?



Inverse Warping

- Get each pixel $g(\mathbf{x}')$ from its corresponding location $\mathbf{x}' = \mathbf{h}(\mathbf{x})$ in $f(\mathbf{x})$
- What if pixel comes from “between” two pixels?
- Answer: *resample* color value from *interpolated* (*prefiltered*) source image



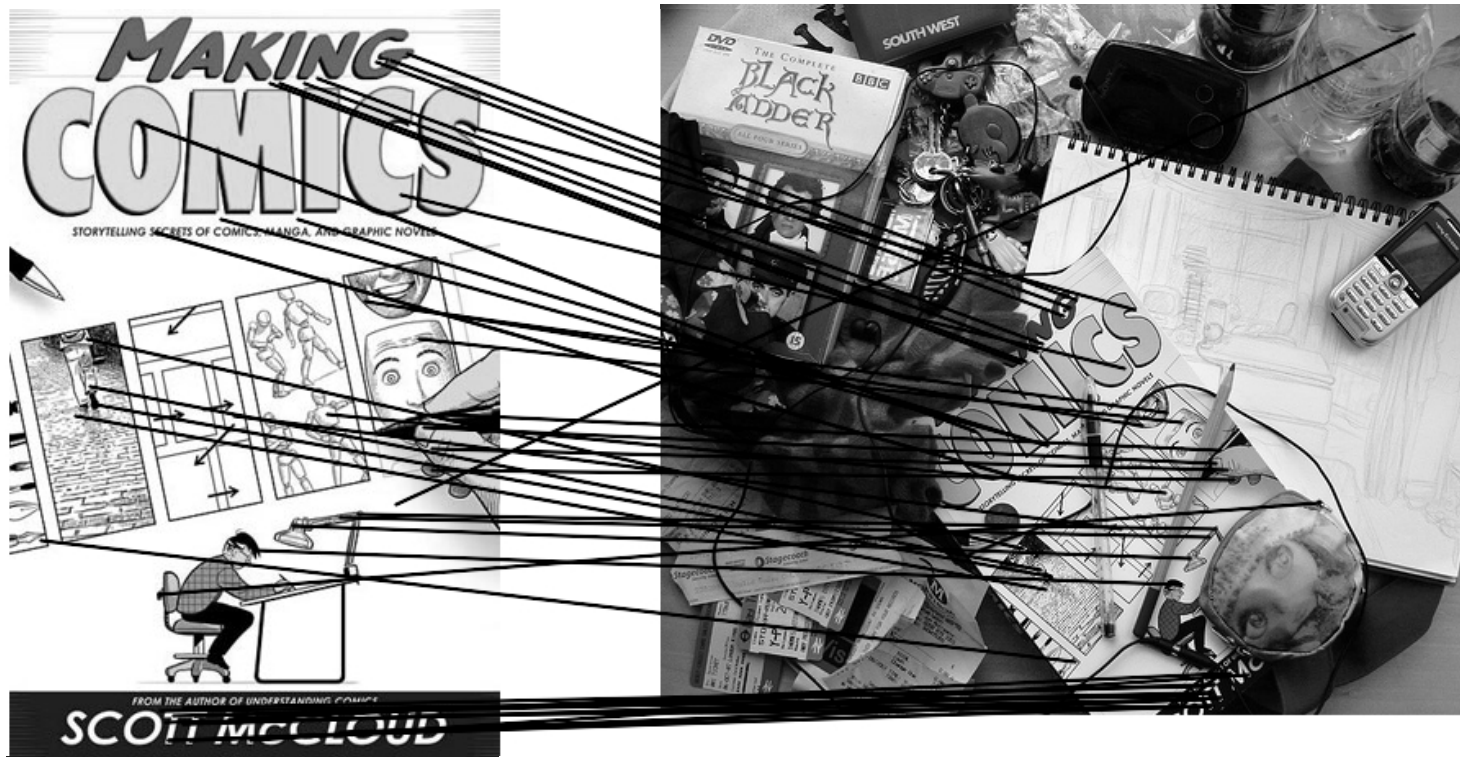
Interpolation

- Possible interpolation filters:
 - nearest neighbor
 - bilinear
 - bicubic (interpolating)
- Needed to prevent “jaggies”
(with prefiltering)



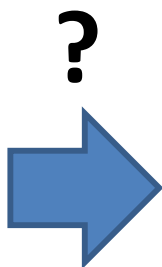
Computing transformations

- Given a set of matches between images A and B
 - How can we compute the transform T from A to B?

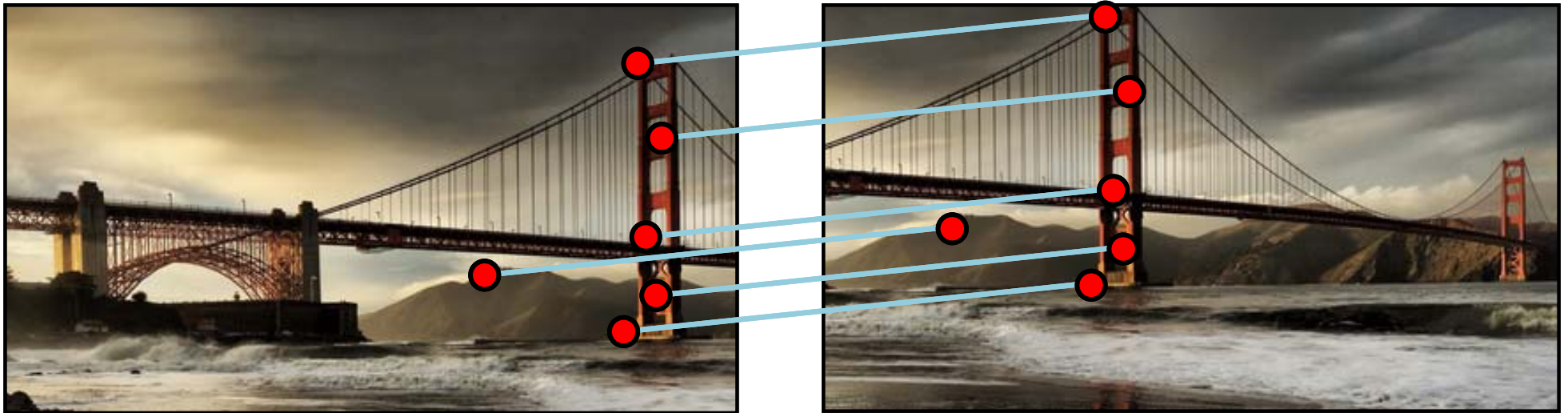


- Find transform T that best “agrees” with the matches

Computing transformations



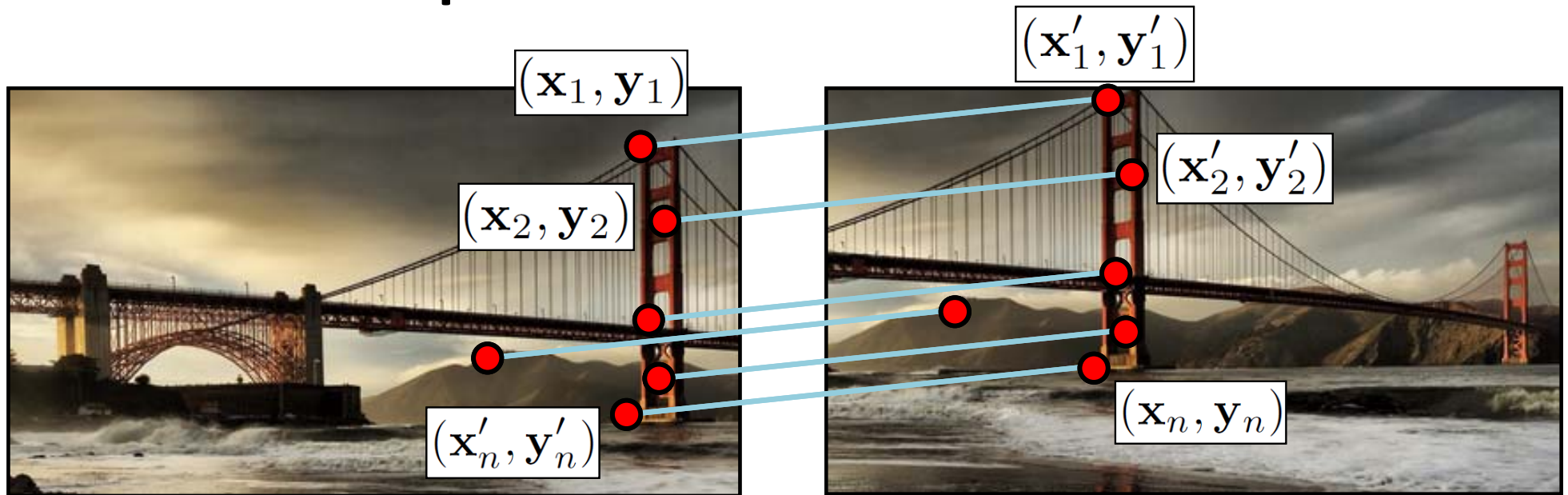
Simple case: translations



How do we solve for
 (x_t, y_t) ?

(x_t, y_t)

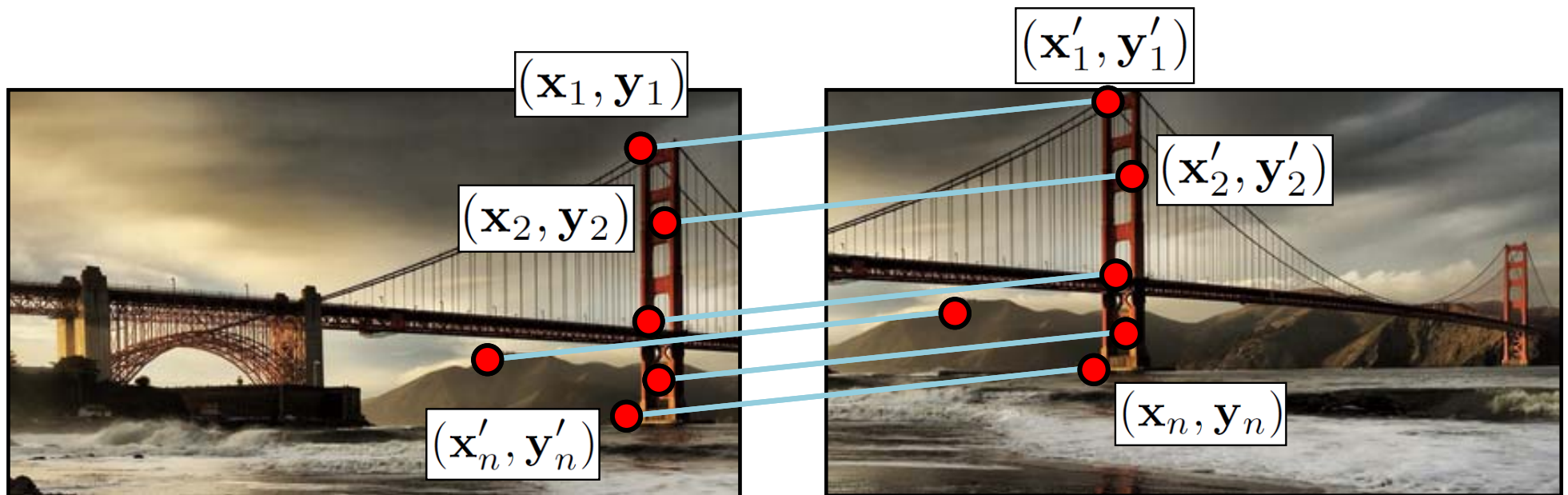
Simple case: translations



Displacement of match $i = (\mathbf{x}'_i - \mathbf{x}_i, \mathbf{y}'_i - \mathbf{y}_i)$

$$(\mathbf{x}_t, \mathbf{y}_t) = \left(\frac{1}{n} \sum_{i=1}^n \mathbf{x}'_i - \mathbf{x}_i, \frac{1}{n} \sum_{i=1}^n \mathbf{y}'_i - \mathbf{y}_i \right)$$

Another view

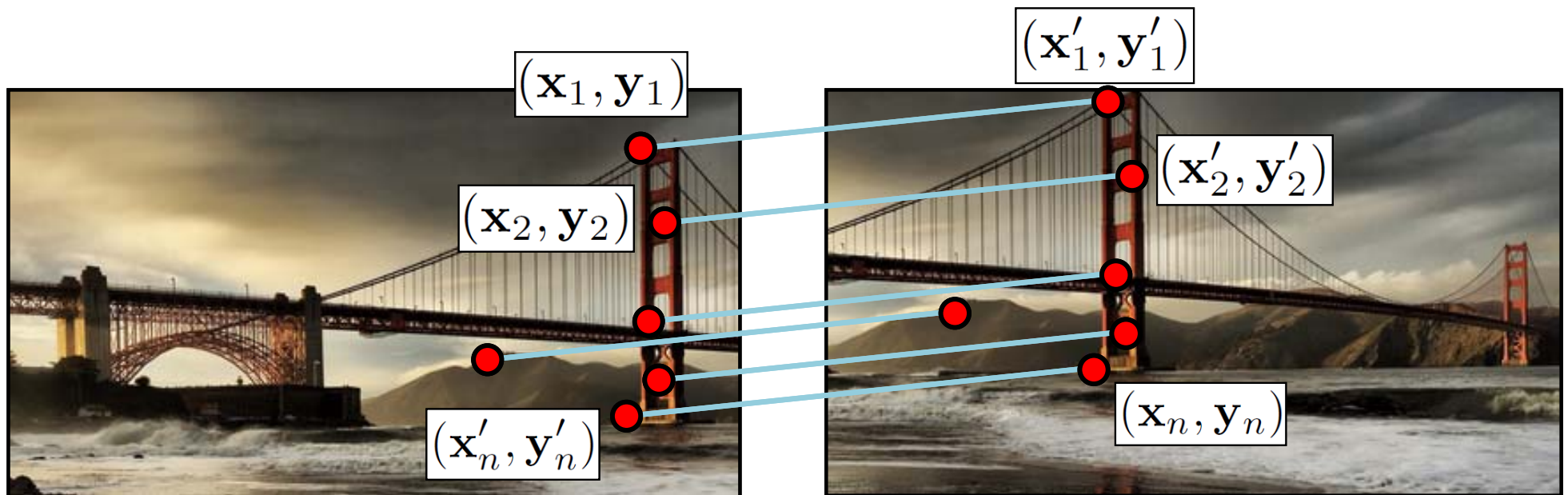


$$\mathbf{x}_i + \mathbf{x}_t = \mathbf{x}'_i$$

$$\mathbf{y}_i + \mathbf{y}_t = \mathbf{y}'_i$$

- System of linear equations
 - What are the knowns? Unknowns?
 - How many unknowns? How many equations (per match)?

Another view



$$\mathbf{x}_i + \mathbf{x}_t = \mathbf{x}'_i$$

$$\mathbf{y}_i + \mathbf{y}_t = \mathbf{y}'_i$$

- Problem: more equations than unknowns
 - “Overdetermined” system of equations
 - We will find the *least squares* solution

Least squares formulation

- For each point $(\mathbf{x}_i, \mathbf{y}_i)$

$$\mathbf{x}_i + \mathbf{x}_t = \mathbf{x}'_i$$

$$\mathbf{y}_i + \mathbf{y}_t = \mathbf{y}'_i$$

- we define the *residuals* as

$$r_{\mathbf{x}_i}(\mathbf{x}_t) = (\mathbf{x}_i + \mathbf{x}_t) - \mathbf{x}'_i$$

$$r_{\mathbf{y}_i}(\mathbf{y}_t) = (\mathbf{y}_i + \mathbf{y}_t) - \mathbf{y}'_i$$

Least squares formulation

- Goal: minimize sum of squared residuals

$$C(\mathbf{x}_t, \mathbf{y}_t) = \sum_{i=1}^n \left(r_{\mathbf{x}_i}(\mathbf{x}_t)^2 + r_{\mathbf{y}_i}(\mathbf{y}_t)^2 \right)$$

- “Least squares” solution
- For translations, is equal to mean displacement

Least squares formulation

- Can also write as a matrix equation

$$\begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 0 \\ 0 & 1 \\ \vdots & \\ 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} x_t \\ y_t \end{bmatrix} = \begin{bmatrix} x'_1 - x_1 \\ y'_1 - y_1 \\ x'_2 - x_2 \\ y'_2 - y_2 \\ \vdots \\ x'_n - x_n \\ y'_n - y_n \end{bmatrix}$$

$$\mathbf{A}$$
$$2n \times 2$$

$$\mathbf{t}$$
$$2 \times 1$$

$$=$$

$$\mathbf{b}$$
$$2n \times 1$$

Least squares

$$\mathbf{A}\mathbf{t} = \mathbf{b}$$

- Find $\mathbf{t}$ that minimizes

$$||\mathbf{A}\mathbf{t} - \mathbf{b}||^2$$

- To solve, form the *normal equations*

$$\mathbf{A}^T \mathbf{A} \mathbf{t} = \mathbf{A}^T \mathbf{b}$$

$$\mathbf{t} = (\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}^T \mathbf{b}$$

Least squares

$$A\mathbf{t} = \mathbf{b}$$

```
t = linsolve(A, b);
```

Computing transformations

- Can also think of as fitting a “model” to our data
 - The model is the transformation of a given type, e.g. a translation, affine xform, homography etc.
 - Fitting the model means solving for the parameters that best explain the observed data
 - Usually involves minimizing some objective / cost function

Solving for translations

- Using least squares – one type of cost function

$$\begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 0 \\ 0 & 1 \\ \vdots & \vdots \\ 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} x_t \\ y_t \end{bmatrix} = \begin{bmatrix} x'_1 - x_1 \\ y'_1 - y_1 \\ x'_2 - x_2 \\ y'_2 - y_2 \\ \vdots \\ x'_n - x_n \\ y'_n - y_n \end{bmatrix}$$

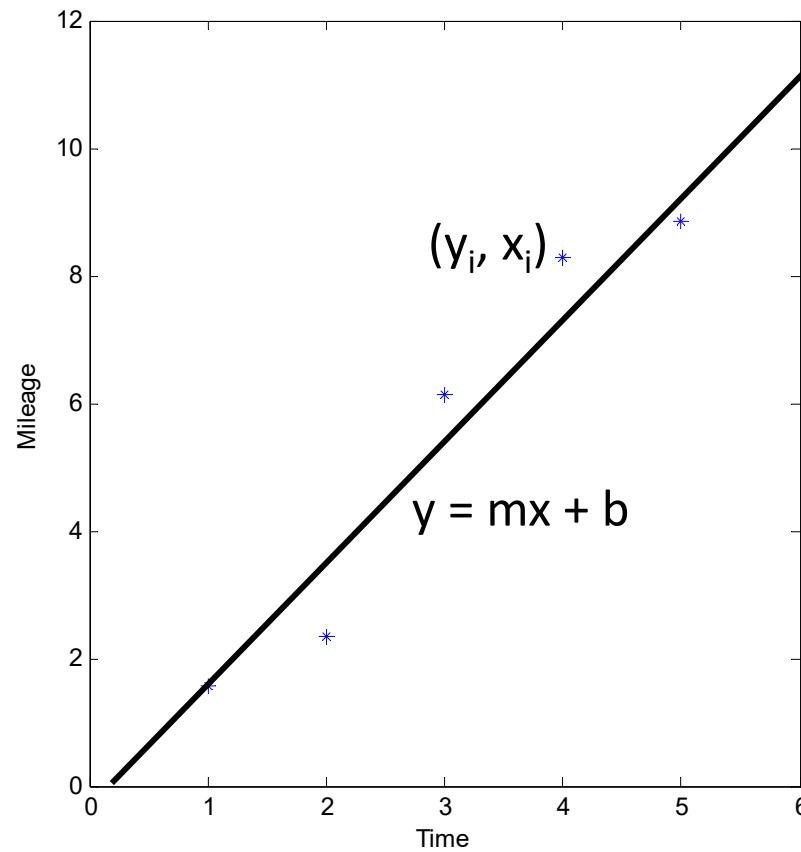
$$\mathbf{A}$$
$$2n \times 2$$

$$\mathbf{t}$$
$$2 \times 1$$

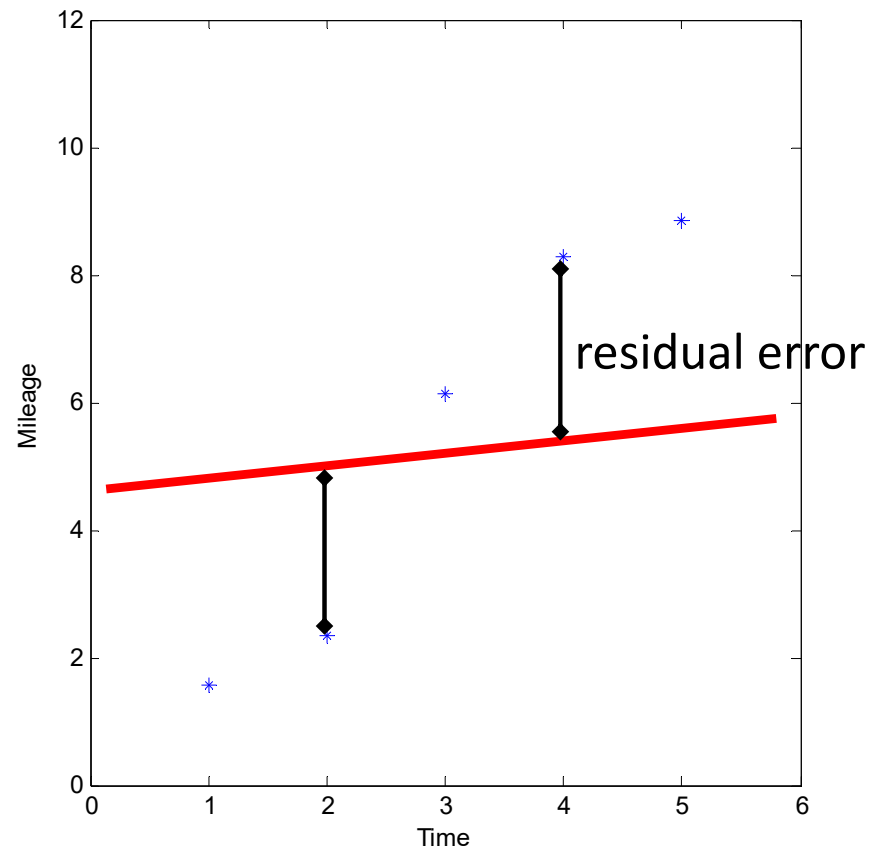
=

$$\mathbf{b}$$
$$2n \times 1$$

Least squares: generalized linear regression



Linear regression



$$\text{Cost}(m, b) = \sum_{i=1}^n |y_i - (mx_i + b)|^2$$

Linear regression

$$\begin{bmatrix} x_1 & 1 \\ x_2 & 1 \\ \vdots & \\ x_n & 1 \end{bmatrix} \begin{bmatrix} m \\ b \end{bmatrix} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}$$

Affine transformations

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} a & b & c \\ d & e & f \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$



- How many unknowns?
- How many equations per match?
- How many matches do we need?

6 unknowns, 2 equations per match → Need at least 3 matches!

Affine transformations

- Residuals:

$$r_{x_i}(a, b, c, d, e, f) = (ax_i + by_i + c) - x'_i$$

$$r_{y_i}(a, b, c, d, e, f) = (dx_i + ey_i + f) - y'_i$$

- Cost function:

$$C(a, b, c, d, e, f) = \sum_{i=1}^n (r_{x_i}(a, b, c, d, e, f)^2 + r_{y_i}(a, b, c, d, e, f)^2)$$

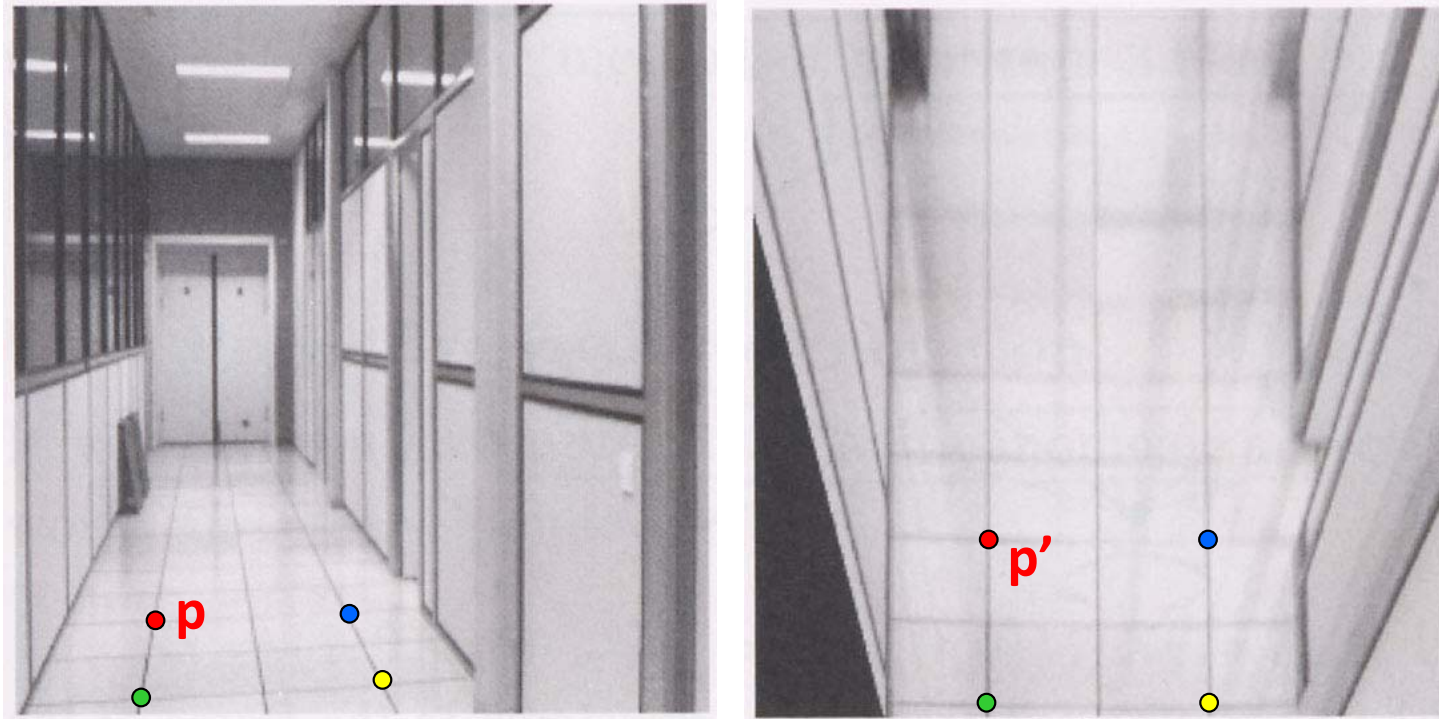
Affine transformations

- Matrix form

$$\begin{bmatrix}
 x_1 & y_1 & 1 & 0 & 0 & 0 \\
 0 & 0 & 0 & x_1 & y_1 & 1 \\
 x_2 & y_2 & 1 & 0 & 0 & 0 \\
 0 & 0 & 0 & x_2 & y_2 & 1 \\
 \vdots & & & & & \\
 x_n & y_n & 1 & 0 & 0 & 0 \\
 0 & 0 & 0 & x_n & y_n & 1
 \end{bmatrix}
 \begin{bmatrix}
 a \\
 b \\
 c \\
 d \\
 e \\
 f
 \end{bmatrix}
 =
 \begin{bmatrix}
 x'_1 \\
 y'_1 \\
 x'_2 \\
 y'_2 \\
 \vdots \\
 x'_n \\
 y'_n
 \end{bmatrix}$$

$$\mathbf{A}_{2n \times 6} \mathbf{t}_{6 \times 1} = \mathbf{b}_{2n \times 1}$$

Homographies



To unwarp (rectify) an image

- solve for homography $\mathbf{H}$ given $\mathbf{p}$ and $\mathbf{p}'$
- solve equations of the form: $w\mathbf{p}' = \mathbf{H}\mathbf{p}$
 - linear in unknowns: w and coefficients of $\mathbf{H}$
 - $\mathbf{H}$ is defined up to an arbitrary scale factor
 - how many points are necessary to solve for $\mathbf{H}$?

Why is this true?

Solving for homographies

$$\begin{bmatrix} x'_i \\ y'_i \\ 1 \end{bmatrix} \cong \begin{bmatrix} h_{00} & h_{01} & h_{02} \\ h_{10} & h_{11} & h_{12} \\ h_{20} & h_{21} & h_{22} \end{bmatrix} \begin{bmatrix} x_i \\ y_i \\ 1 \end{bmatrix}$$

$$x'_i = \frac{h_{00}x_i + h_{01}y_i + h_{02}}{h_{20}x_i + h_{21}y_i + h_{22}}$$

$$y'_i = \frac{h_{10}x_i + h_{11}y_i + h_{12}}{h_{20}x_i + h_{21}y_i + h_{22}}$$

Not linear!

$$x'_i(h_{20}x_i + h_{21}y_i + h_{22}) = h_{00}x_i + h_{01}y_i + h_{02}$$

$$y'_i(h_{20}x_i + h_{21}y_i + h_{22}) = h_{10}x_i + h_{11}y_i + h_{12}$$

Solving for homographies

$$x'_i(h_{20}x_i + h_{21}y_i + h_{22}) = h_{00}x_i + h_{01}y_i + h_{02}$$

$$y'_i(h_{20}x_i + h_{21}y_i + h_{22}) = h_{10}x_i + h_{11}y_i + h_{12}$$

$$\begin{bmatrix} x_i & y_i & 1 & 0 & 0 & 0 & -x'_i x_i & -x'_i y_i & -x'_i \\ 0 & 0 & 0 & x_i & y_i & 1 & -y'_i x_i & -y'_i y_i & -y'_i \end{bmatrix} \begin{bmatrix} h_{00} \\ h_{01} \\ h_{02} \\ h_{10} \\ h_{11} \\ h_{12} \\ h_{20} \\ h_{21} \\ h_{22} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

Solving for homographies

$$\begin{bmatrix}
 x_1 & y_1 & 1 & 0 & 0 & 0 & -x'_1 x_1 & -x'_1 y_1 & -x'_1 \\
 0 & 0 & 0 & x_1 & y_1 & 1 & -y'_1 x_1 & -y'_1 y_1 & -y'_1 \\
 & & & & & \vdots & & & \\
 x_n & y_n & 1 & 0 & 0 & 0 & -x'_n x_n & -x'_n y_n & -x'_n \\
 0 & 0 & 0 & x_n & y_n & 1 & -y'_n x_n & -y'_n y_n & -y'_n
 \end{bmatrix}
 \begin{bmatrix}
 h_{00} \\
 h_{01} \\
 h_{02} \\
 h_{10} \\
 h_{11} \\
 h_{12} \\
 h_{20} \\
 h_{21} \\
 h_{22}
 \end{bmatrix}
 =
 \begin{bmatrix}
 0 \\
 0 \\
 \vdots \\
 0 \\
 0
 \end{bmatrix}$$

$\mathbf{A}$
 $2n \times 9$

$\mathbf{h}$
 9

$\mathbf{0}$
 $2n$

Defines a least squares problem: minimize $\|\mathbf{A}\mathbf{h} - \mathbf{0}\|^2$

- Since $\mathbf{h}$ is only defined up to scale, solve for unit vector $\hat{\mathbf{h}}$
- Solution: $\hat{\mathbf{h}}$ = eigenvector of $\mathbf{A}^T \mathbf{A}$ with smallest eigenvalue
- Works with 4 or more points

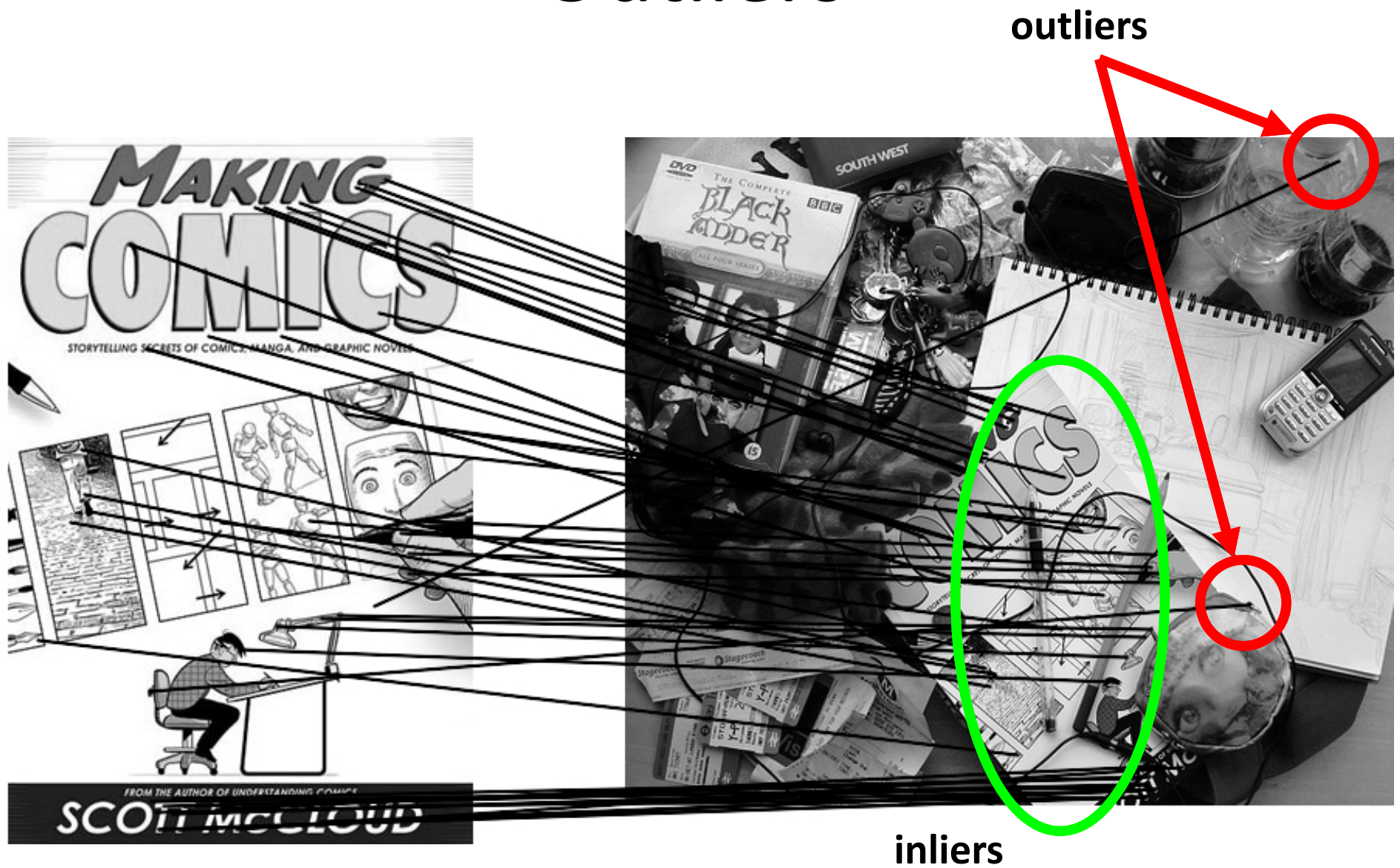
Image Alignment Algorithm

Given images A and B

1. Compute image features for A and B
2. Match features between A and B
3. Compute homography between A and B using least squares on set of matches

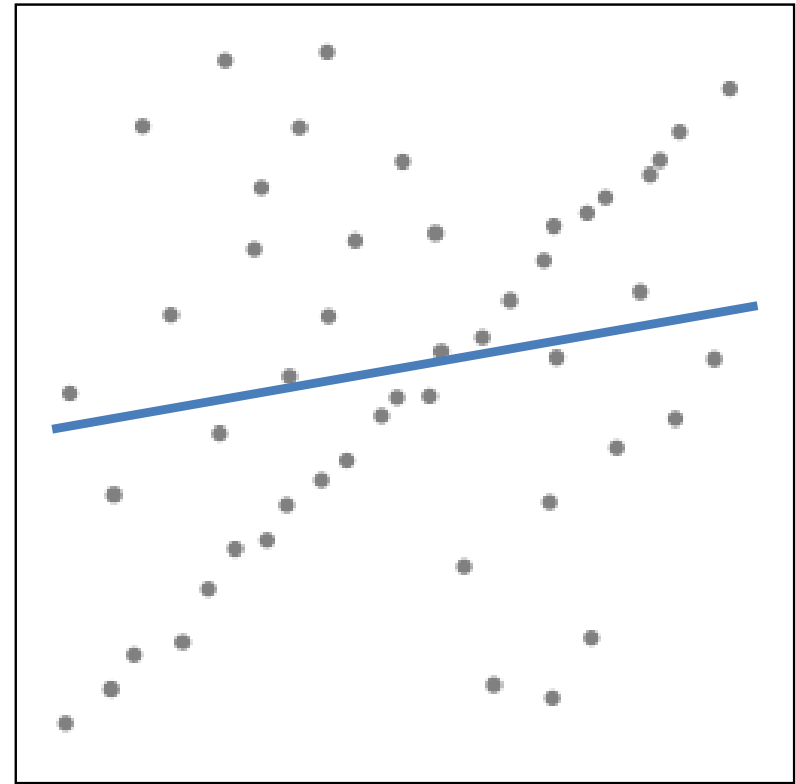
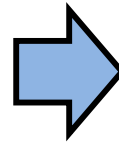
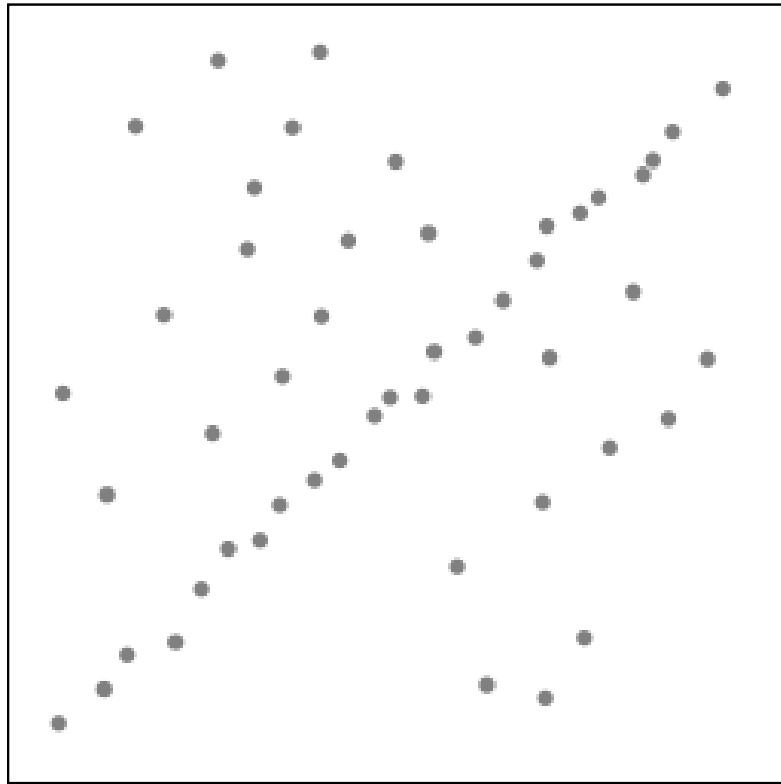
What could go wrong?

Outliers



Robustness

- Let's consider a simpler example... linear regression



Problem: Fit a line to these datapoints

Least squares fit

- How can we fix this?

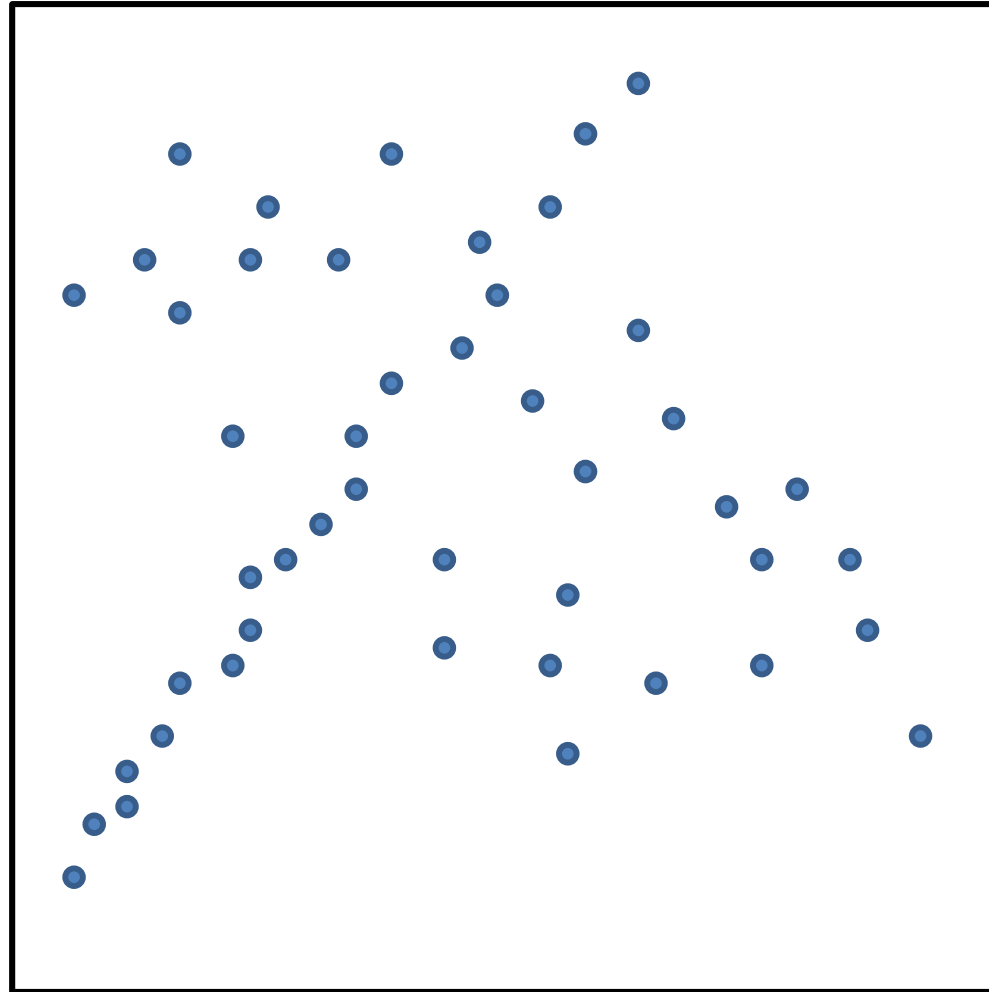
We need a better cost function...

- Suggestions?

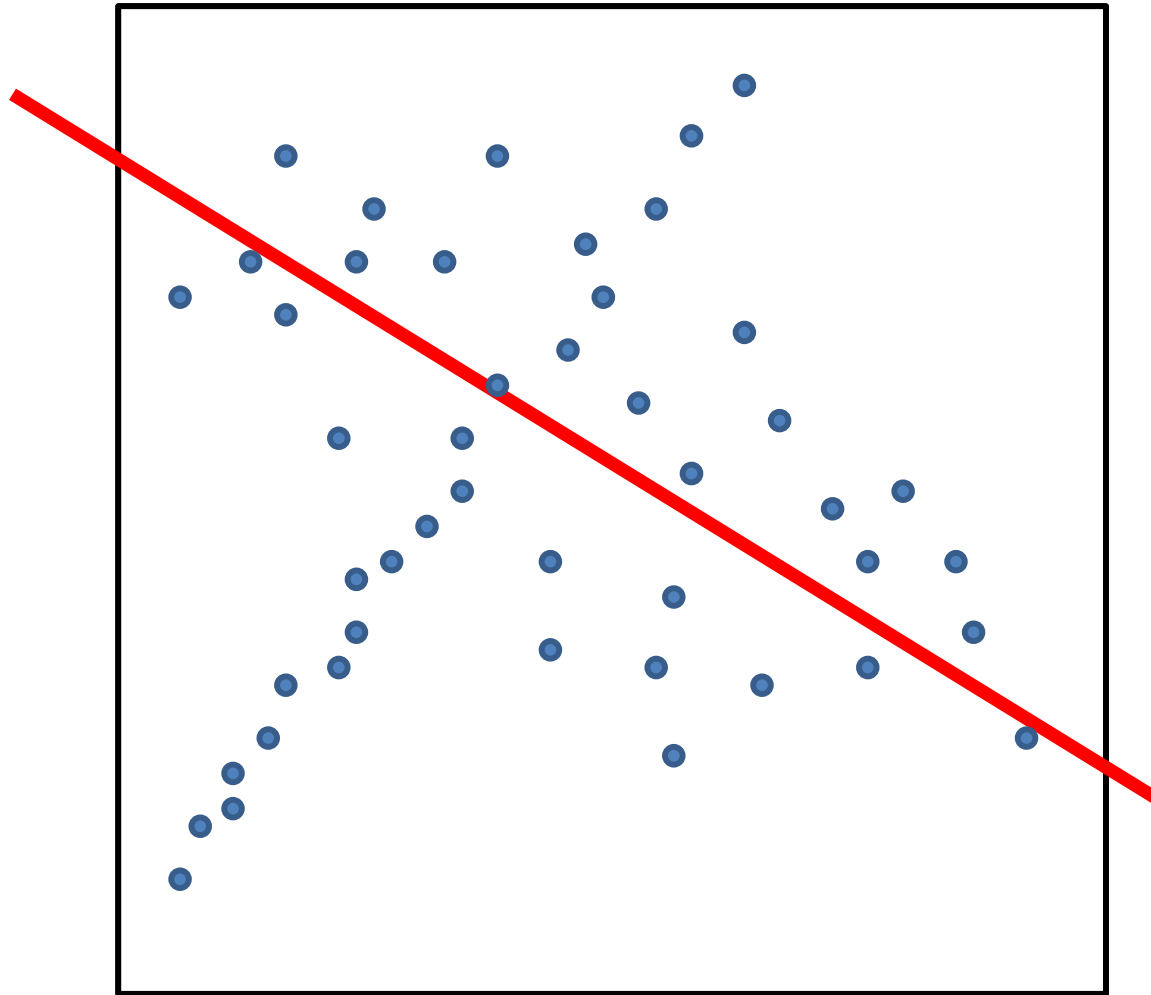
Idea

- Given a hypothesized line
- Count the number of points that “agree” with the line
 - “Agree” = within a small distance of the line
 - I.e., the **inliers** to that line
- For all possible lines, select the one with the largest number of inliers

Counting inliers

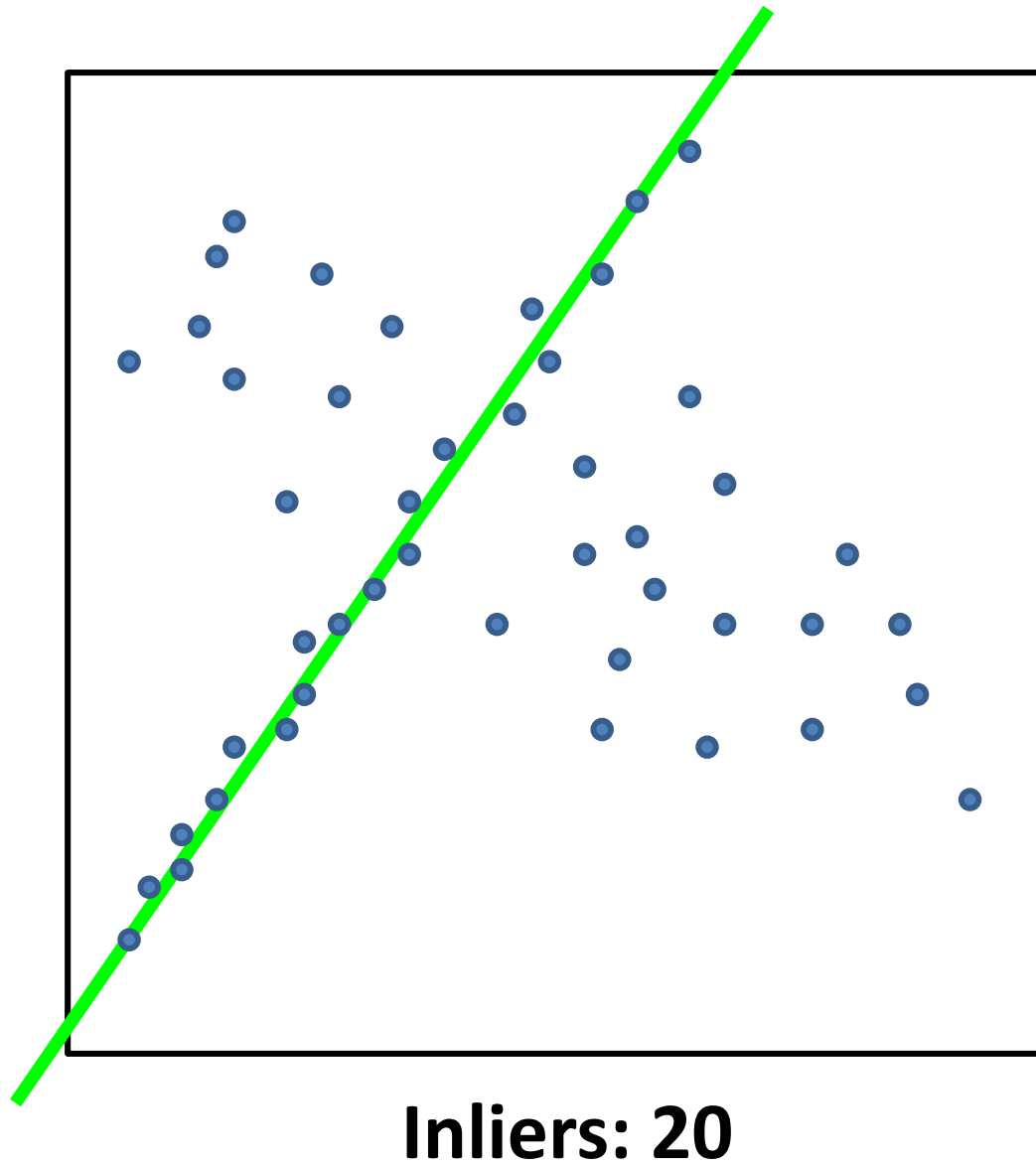


Counting inliers



Inliers: 3

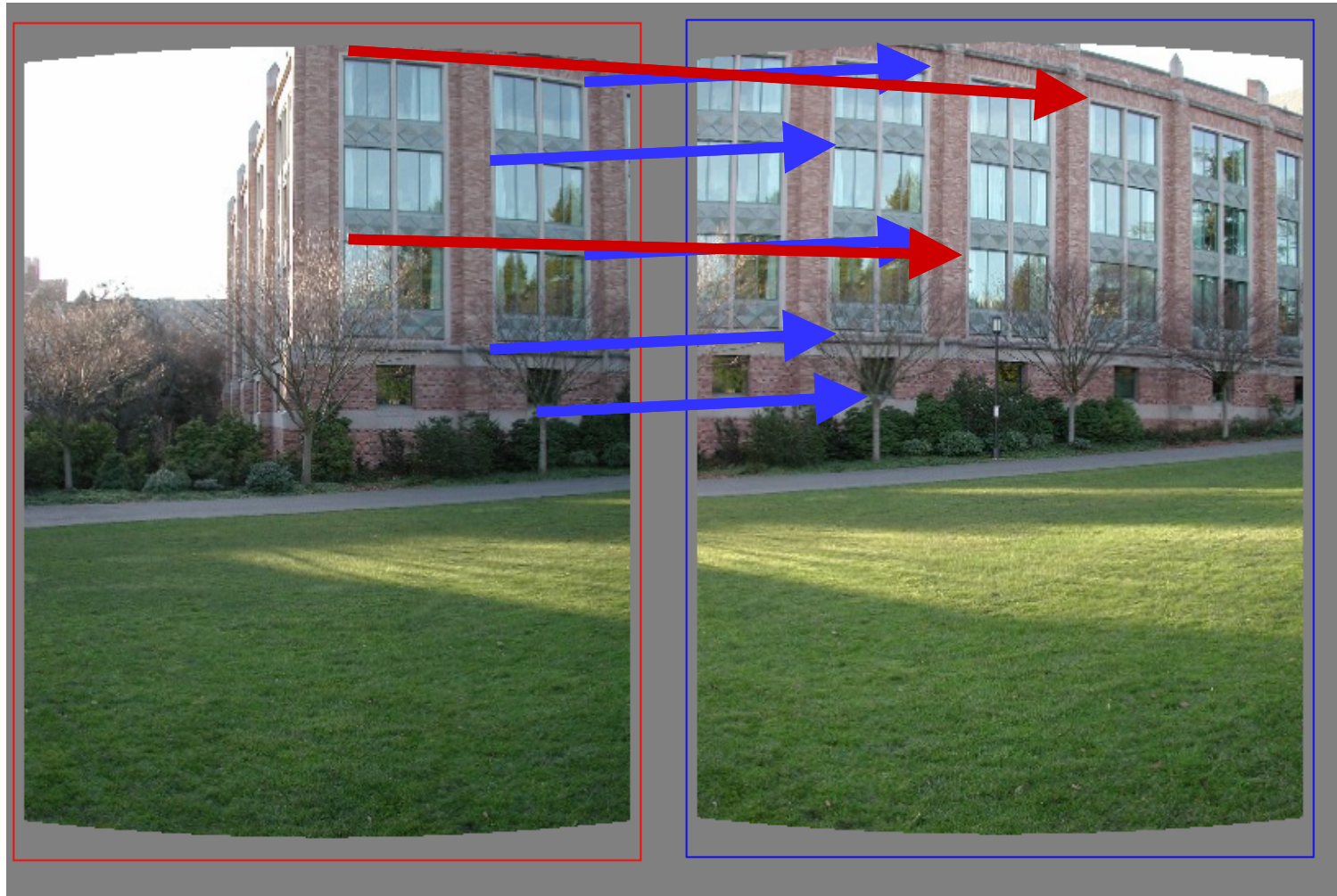
Counting inliers



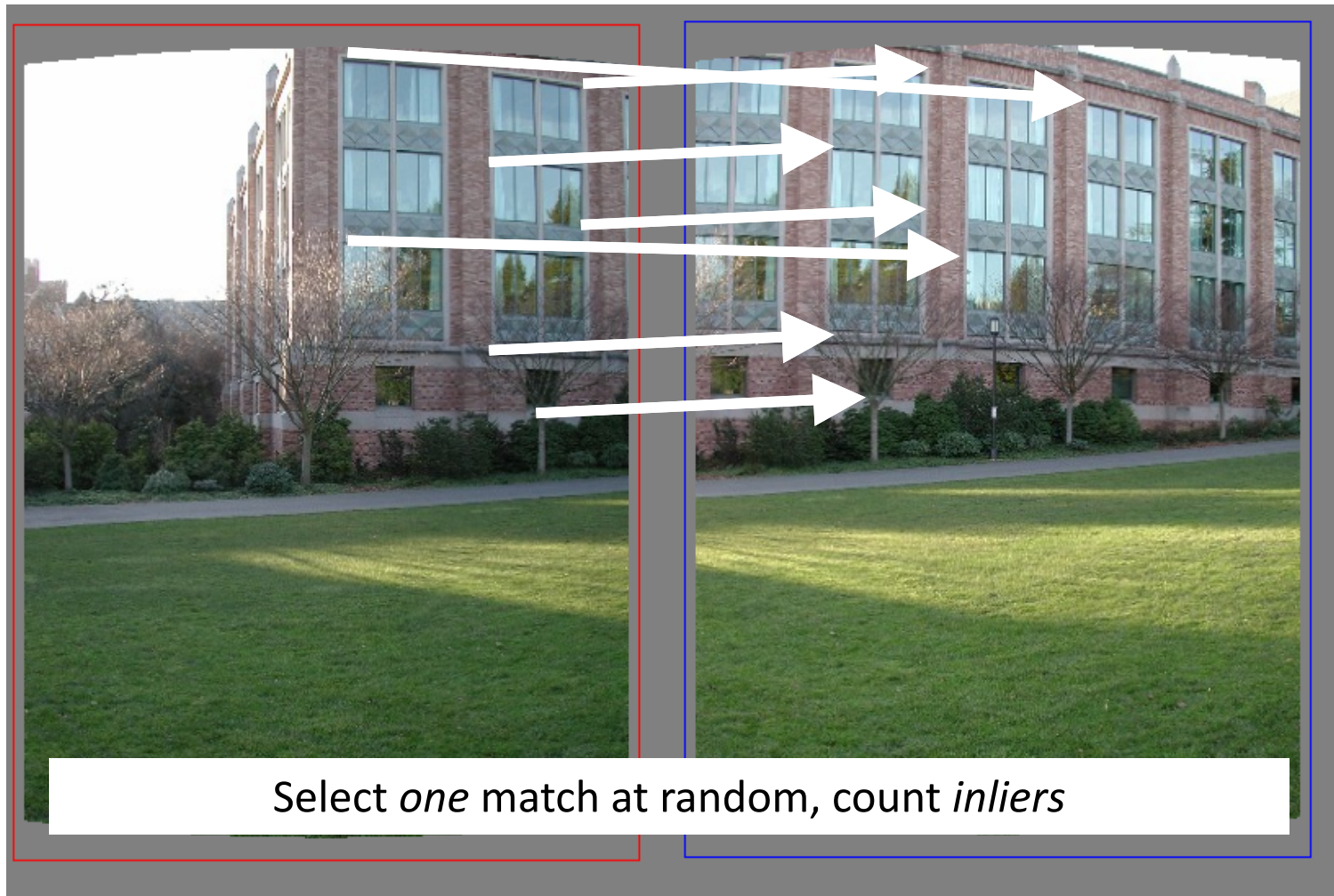
How do we find the best line?

- Unlike least-squares, no simple closed-form solution
- Hypothesize-and-test
 - Try out many lines, keep the best one
 - Which lines?

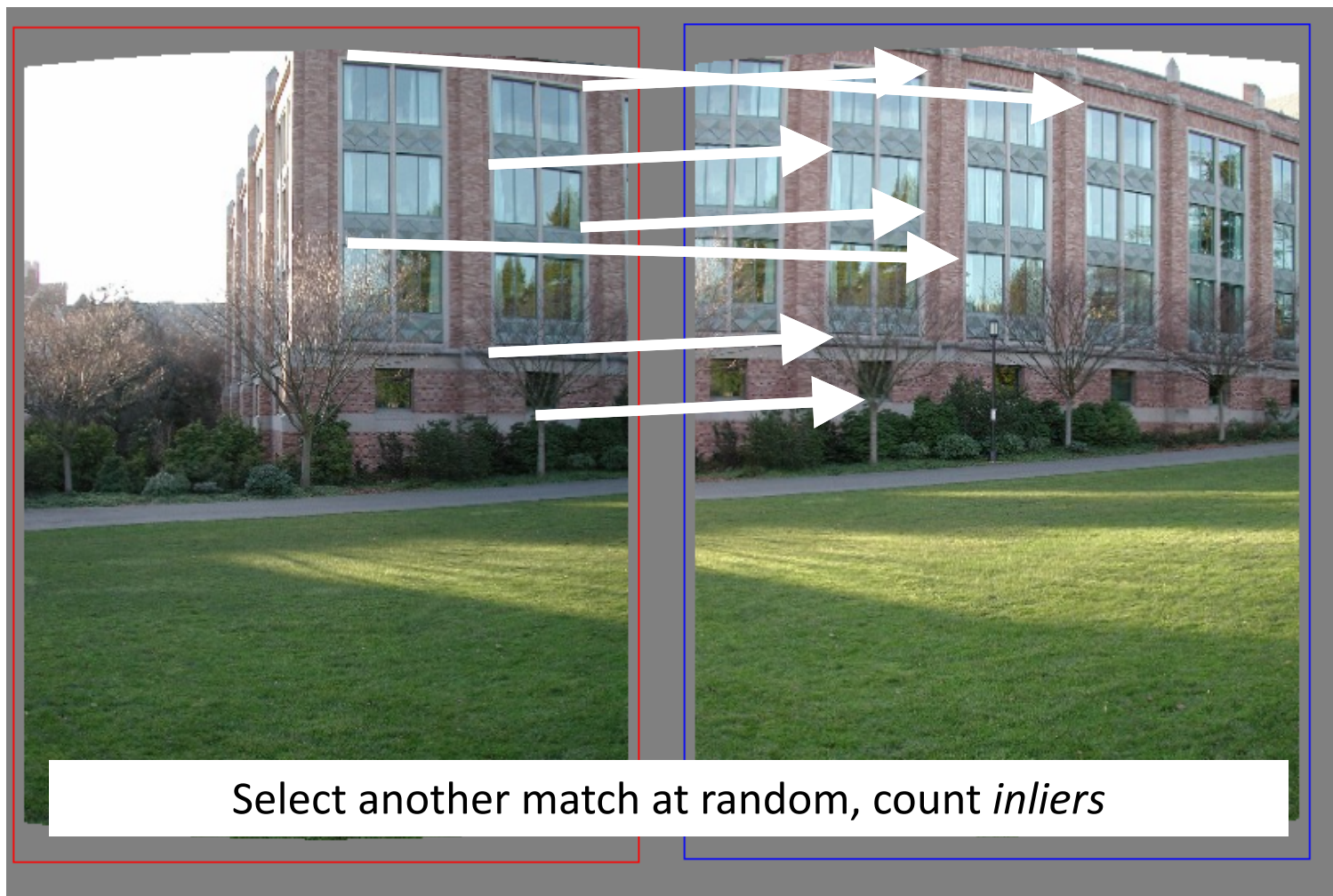
Translations



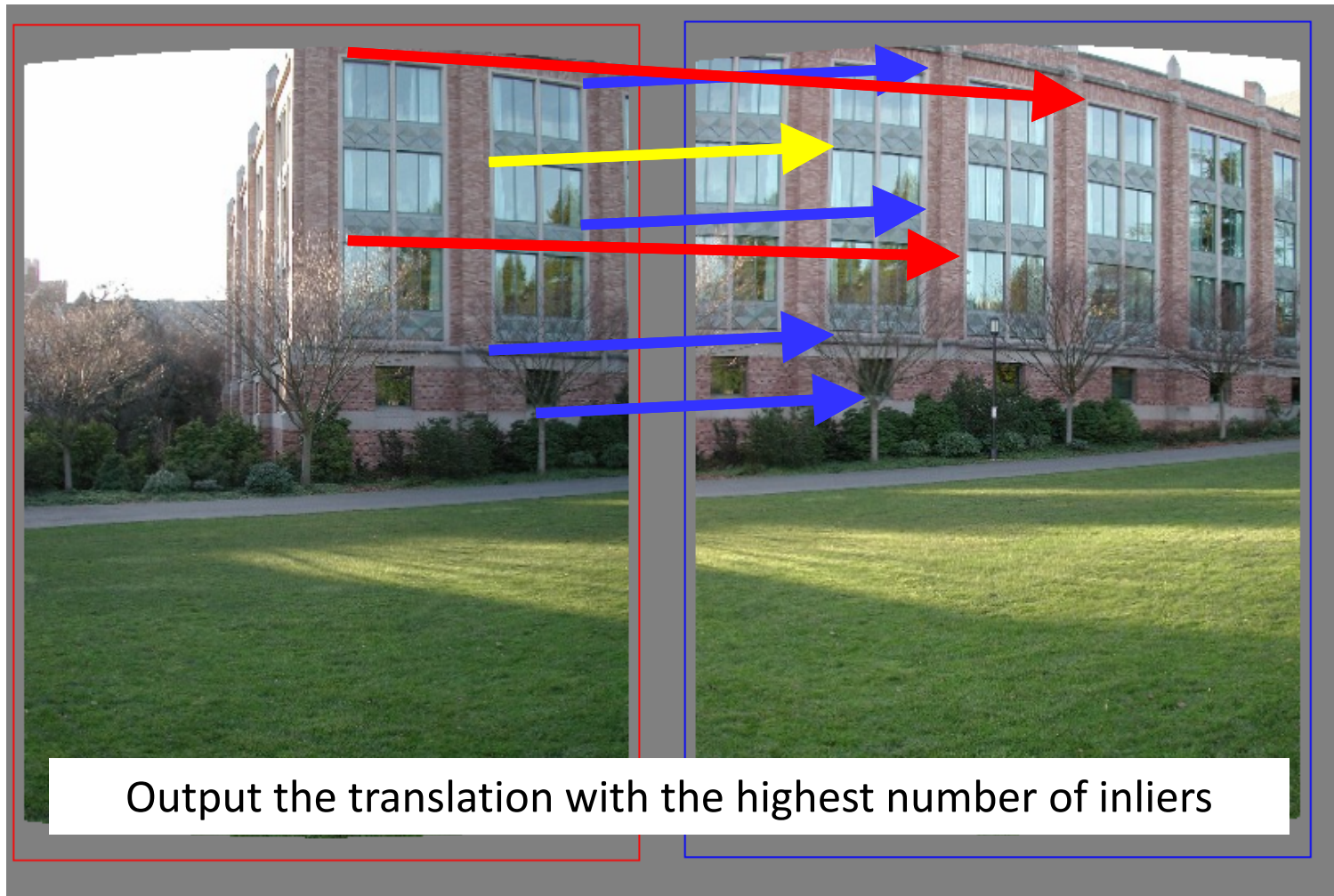
Random Sample Consensus



Random Sample Consensus



Random Sample Consensus



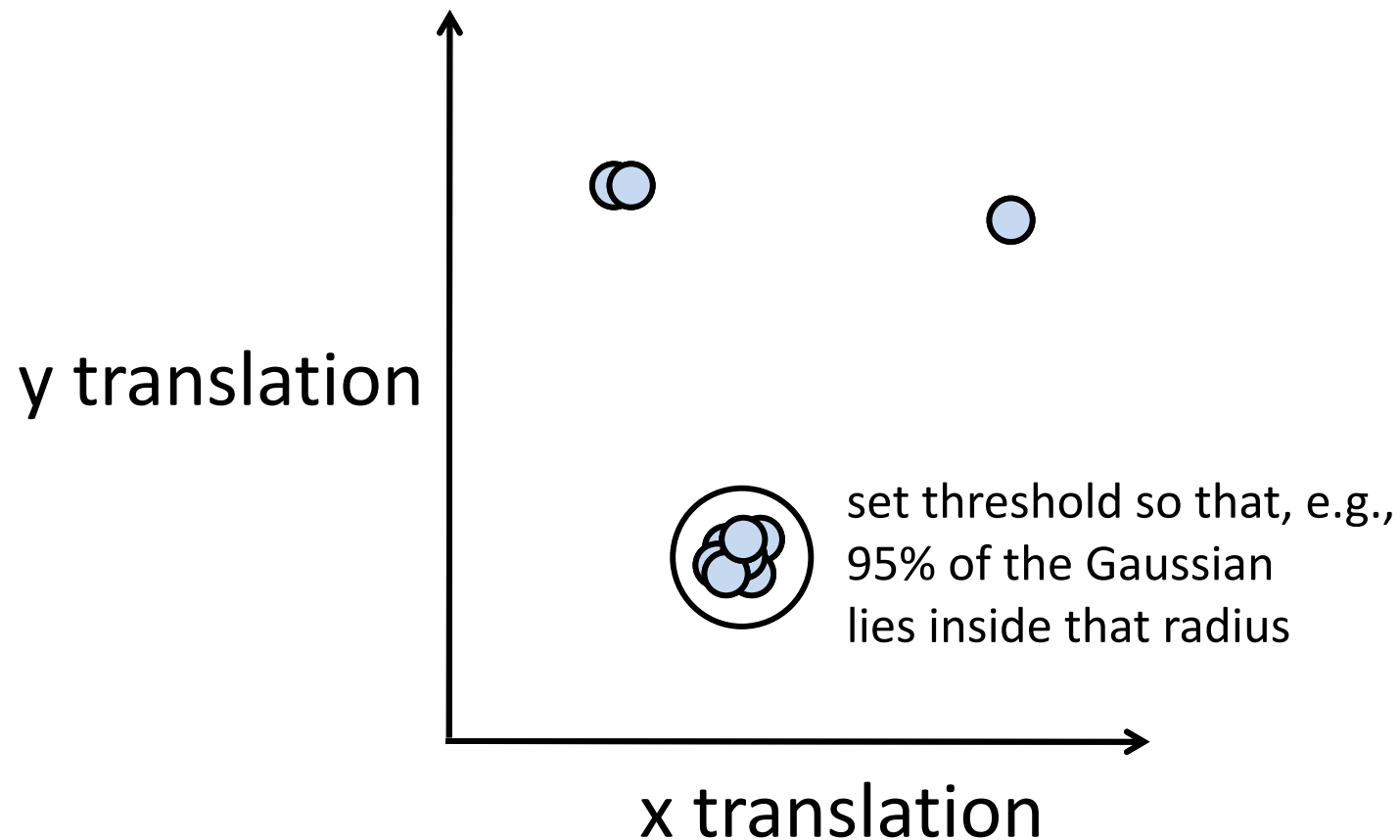
RANSAC

- Idea:
 - All the inliers will agree with each other on the translation vector; the (hopefully small) number of outliers will (hopefully) disagree with each other
 - RANSAC only has guarantees if there are $< 50\%$ outliers
 - “All good matches are alike; every bad match is bad in its own way.”
 - Tolstoy via Alyosha Efros

RANSAC

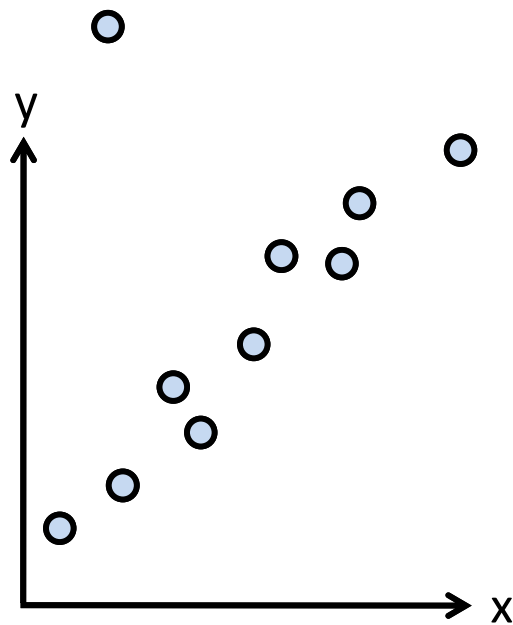
- **Inlier threshold** related to the amount of noise we expect in inliers
 - Often model noise as Gaussian with some standard deviation (e.g., 3 pixels)
- **Number of rounds** related to the percentage of outliers we expect, and the probability of success we'd like to guarantee
 - Suppose there are 20% outliers, and we want to find the correct answer with 99% probability
 - How many rounds do we need?

RANSAC



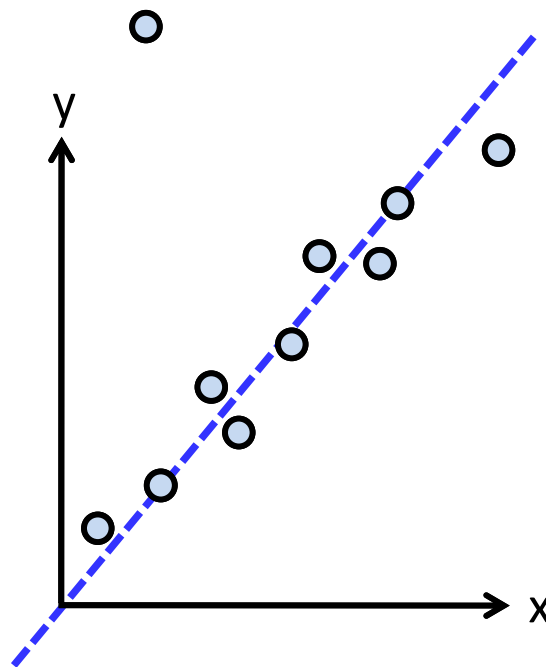
RANSAC

- Back to linear regression
- How do we generate a hypothesis?



RANSAC

- Back to linear regression
- How do we generate a hypothesis?



RANSAC

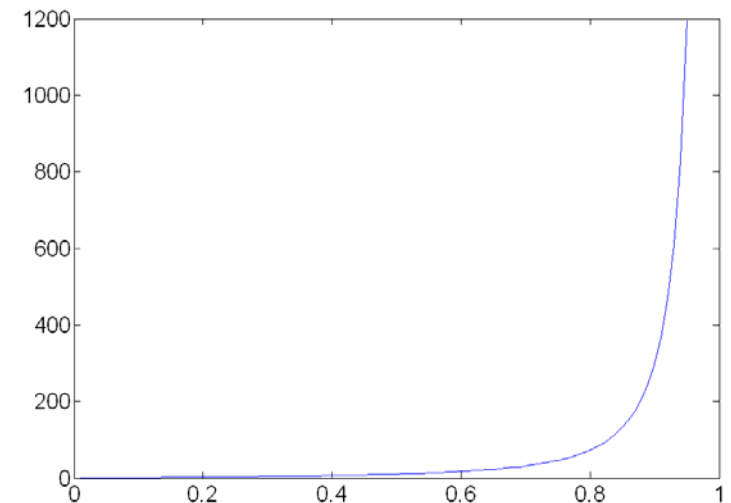
- General version:
 1. Randomly choose s samples
 - Typically s = minimum sample size that lets you fit a model
 2. Fit a model (e.g., line) to those samples
 3. Count the number of inliers that approximately fit the model
 4. Repeat N times
 5. Choose the model that has the largest set of inliers

How many rounds?

- If we have to choose s samples each time
 - with an outlier ratio e
 - and we want the right answer with probability p

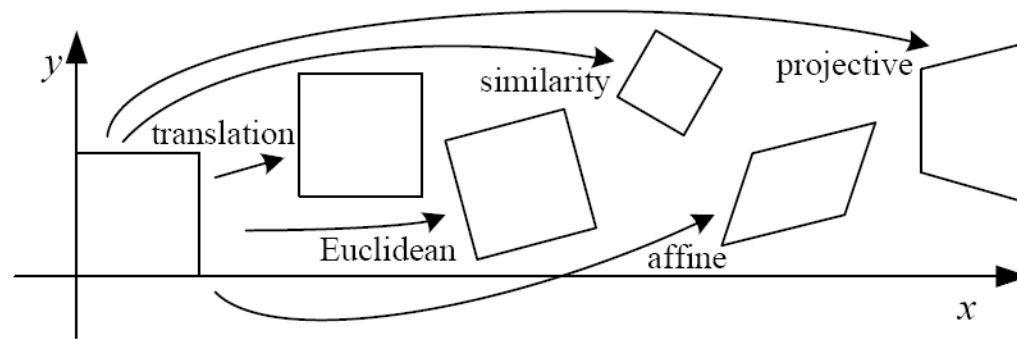
s	proportion of outliers e						
	5%	10%	20%	25%	30%	40%	50%
2	2	3	5	6	7	11	17
3	3	4	7	9	11	19	35
4	3	5	9	13	17	34	72
5	4	6	12	17	26	57	146
6	4	7	16	24	37	97	293
7	4	8	20	33	54	163	588
8	5	9	26	44	78	272	1177

$p = 0.99$



How big is s ?

- For alignment, depends on the motion model
 - Here, each sample is a correspondence (pair of matching points)

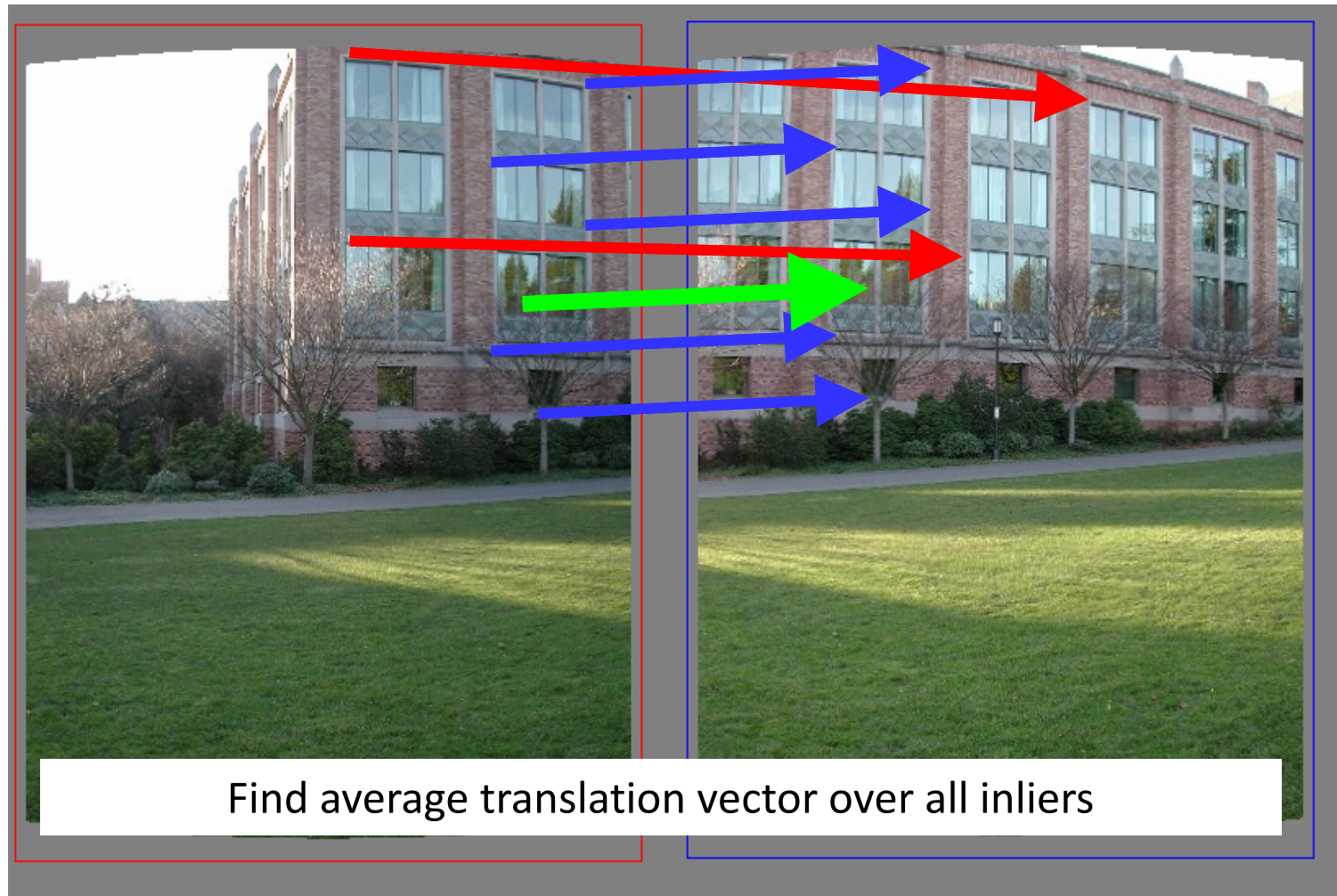


Name	Matrix	# D.O.F.	Preserves:	Icon
translation	$\begin{bmatrix} \mathbf{I} & \mathbf{t} \end{bmatrix}_{2 \times 3}$	2	orientation + ...	
rigid (Euclidean)	$\begin{bmatrix} \mathbf{R} & \mathbf{t} \end{bmatrix}_{2 \times 3}$	3	lengths + ...	
similarity	$\begin{bmatrix} s\mathbf{R} & \mathbf{t} \end{bmatrix}_{2 \times 3}$	4	angles + ...	
affine	$\begin{bmatrix} \mathbf{A} \end{bmatrix}_{2 \times 3}$	6	parallelism + ...	
projective	$\begin{bmatrix} \tilde{\mathbf{H}} \end{bmatrix}_{3 \times 3}$	8	straight lines	

RANSAC pros and cons

- Pros
 - Simple and general
 - Applicable to many different problems
 - Often works well in practice
- Cons
 - Parameters to tune
 - Sometimes too many iterations are required
 - Can fail for extremely low inlier ratios

Final step: least squares fit



RANSAC

- An example of a “voting”-based fitting scheme
- Each hypothesis gets voted on by each data point, best hypothesis wins
- There are many other types of voting schemes
 - E.g., Hough transforms...

Panoramas

- Now we know how to create panoramas!
- Given two images:
 - Step 1: Detect features
 - Step 2: Match features
 - Step 3: Compute a homography using RANSAC
 - Step 4: Combine the images together (somehow)
- What if we have more than two images?

Panorama examples

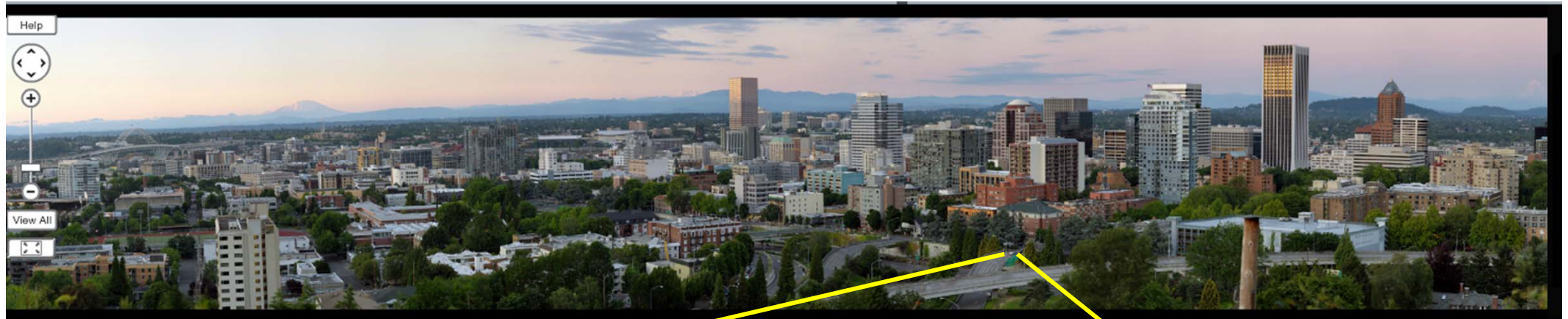
- Every image on Google Streetview



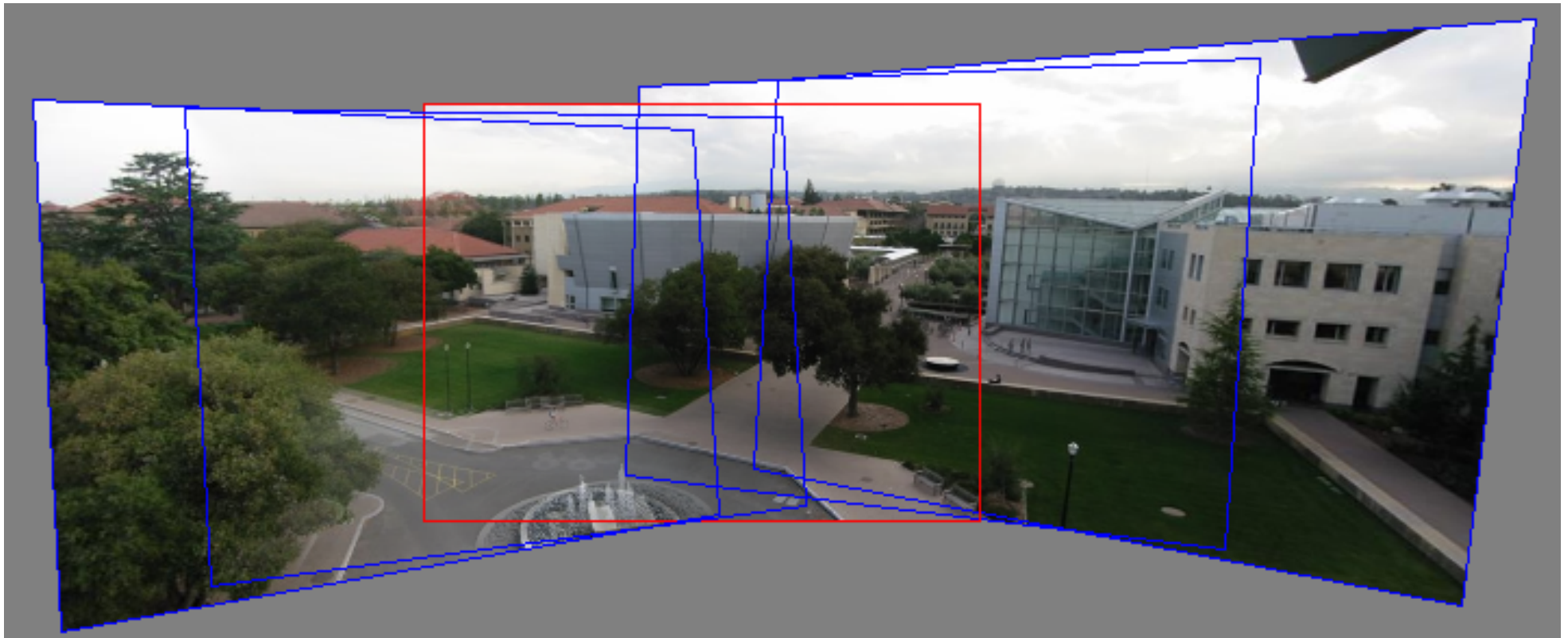
GigaPan



GigaPan



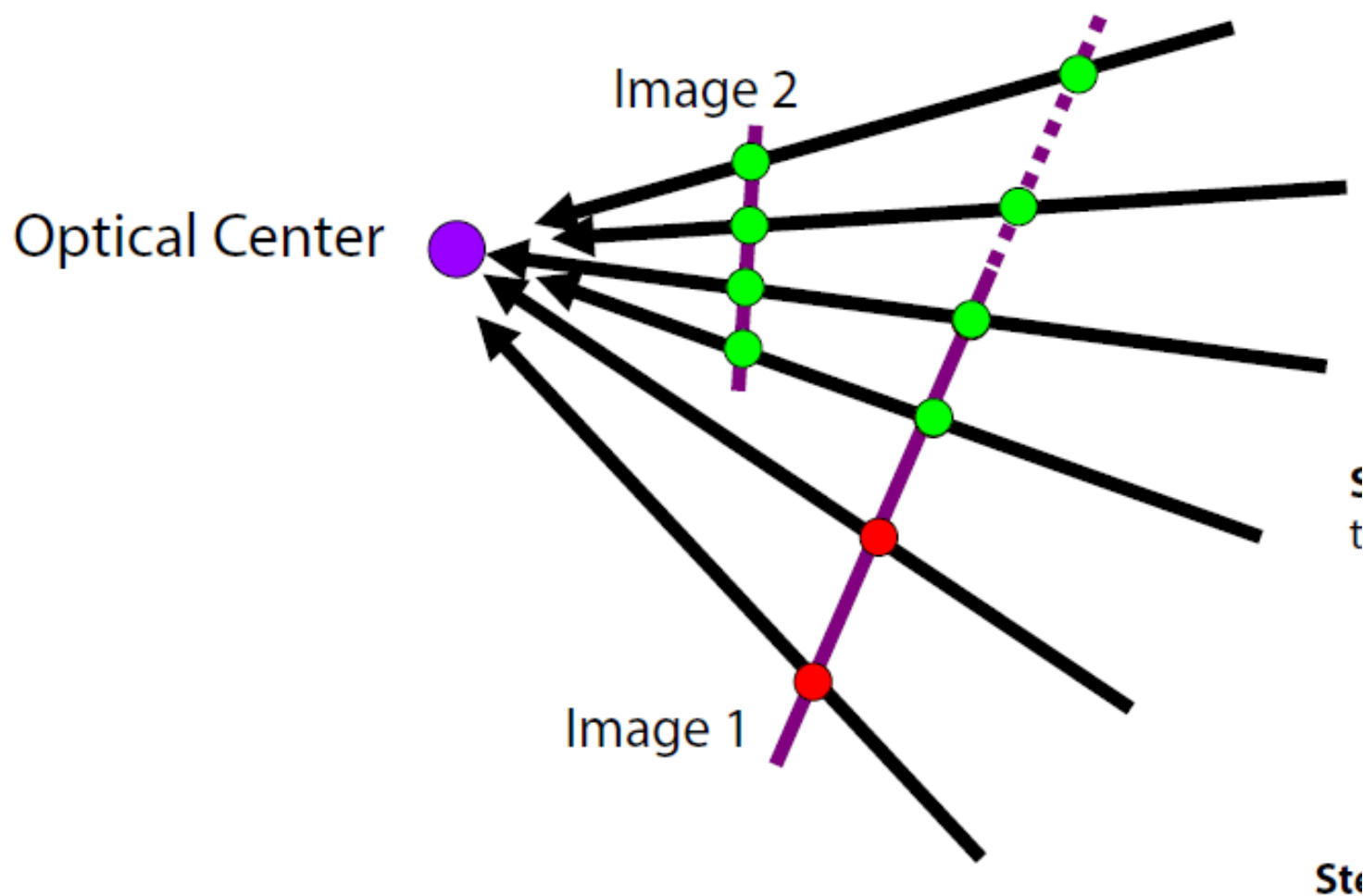
Can we use homographies to create a 360 panorama?



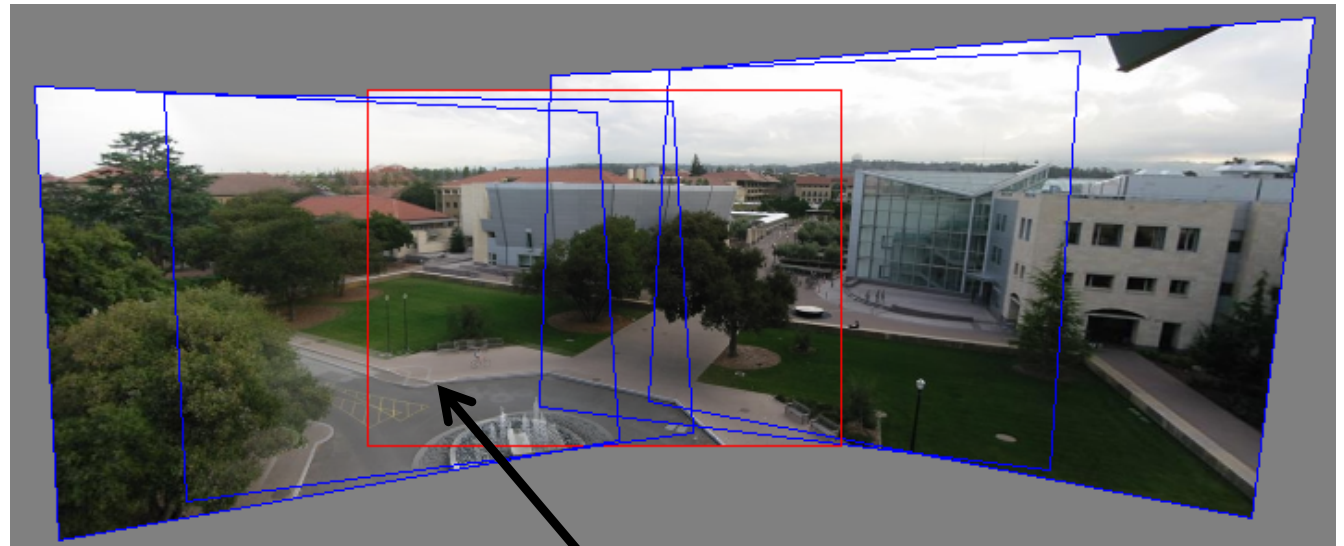
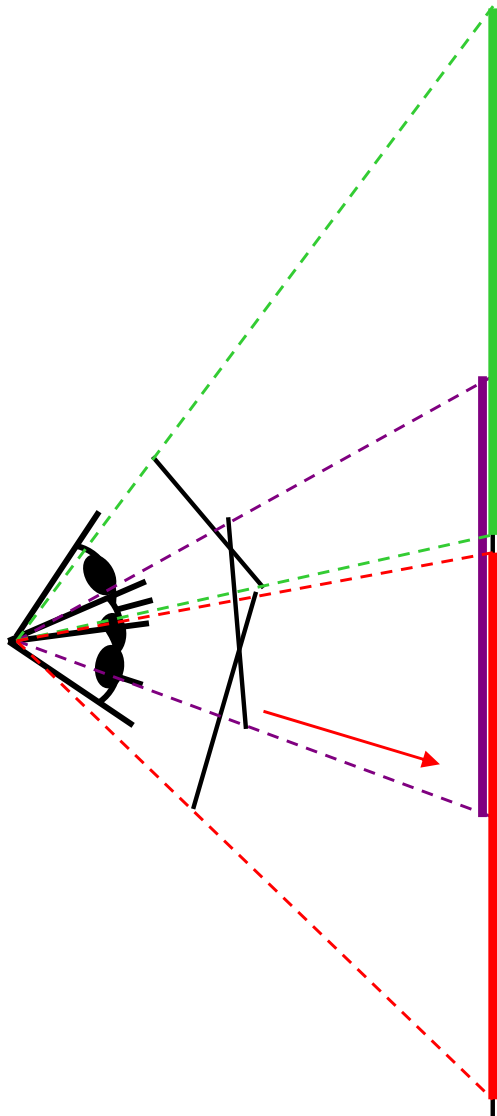
360 panorama



What is the transformation?



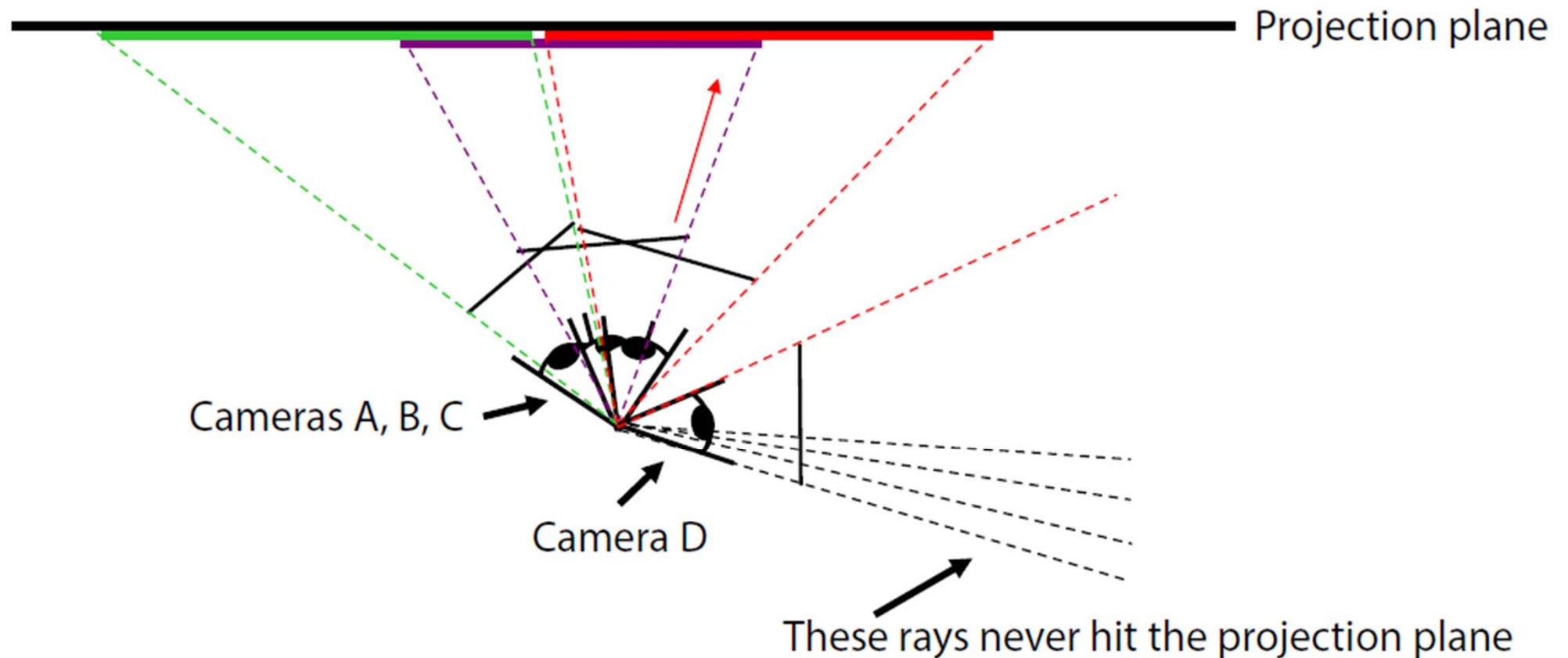
Idea: projecting images onto a common plane



each image is warped
with a homography **H**

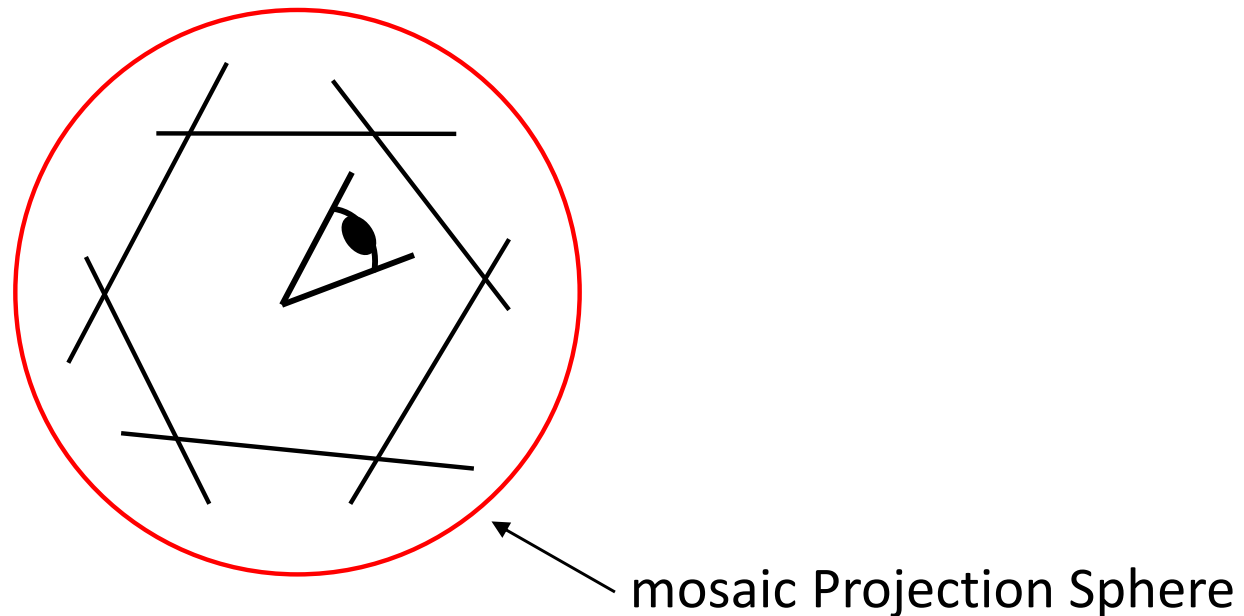
mosaic PP

Can we use homography to create a 360 panorama?

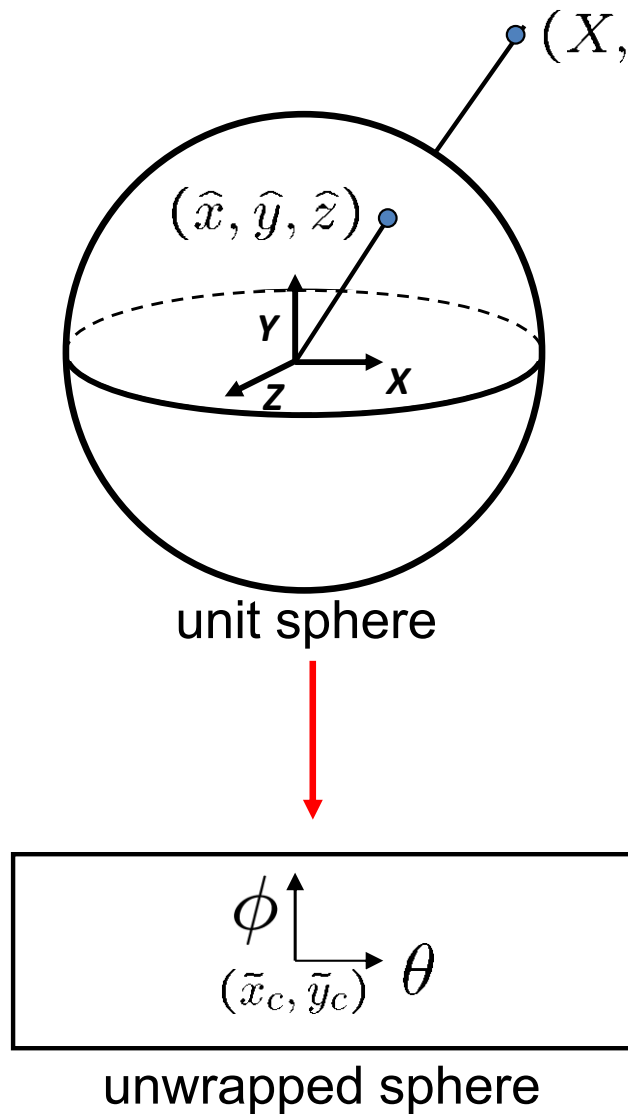


Panoramas

- What if you want a 360° field of view?



Spherical projection



- Map 3D point (X, Y, Z) onto sphere

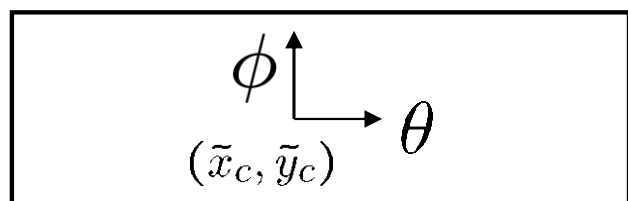
$$(\hat{x}, \hat{y}, \hat{z}) = \frac{1}{\sqrt{X^2 + Y^2 + Z^2}}(X, Y, Z)$$

- Convert to spherical coordinates
- Convert to spherical image coordinates

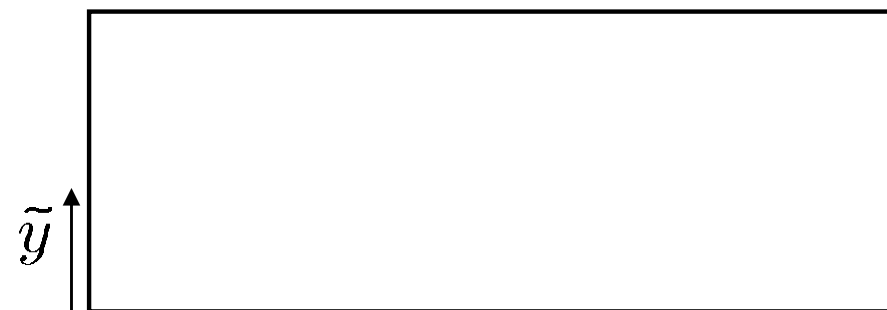
$$(\tilde{x}, \tilde{y}) = (s\theta, s\phi) + (\tilde{x}_c, \tilde{y}_c)$$

- s defines size of the final image
 - » often convenient to set s = camera focal length in pixels

Spherical projection



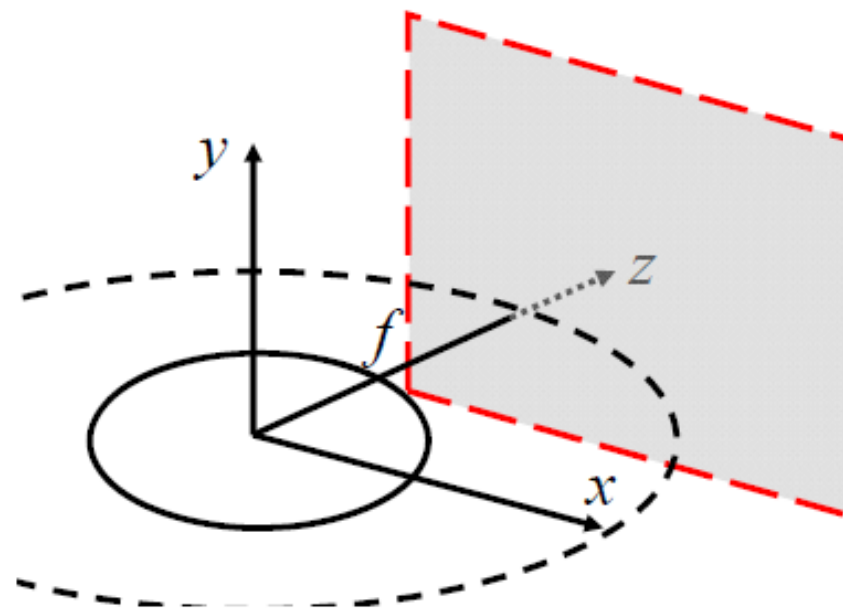
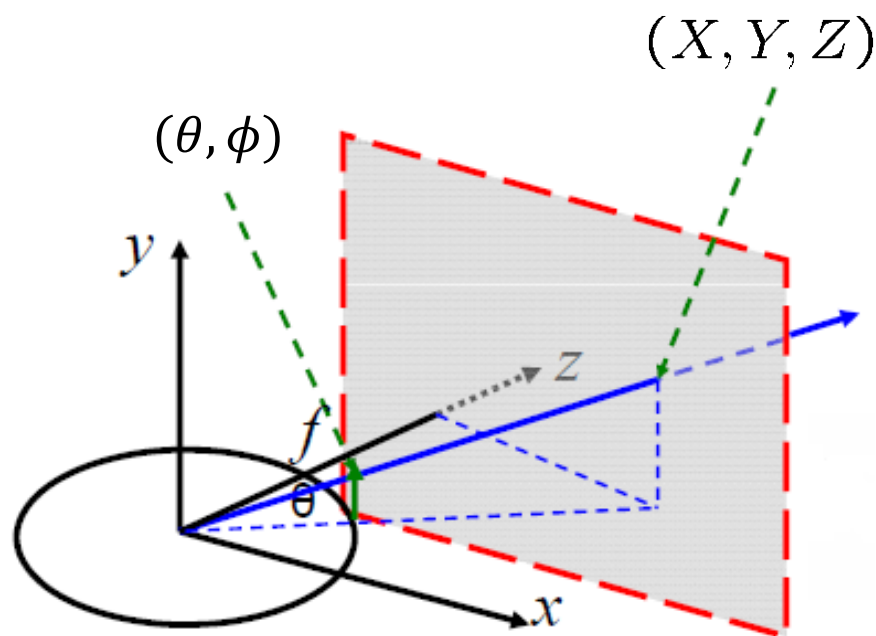
unwrapped sphere



Spherical image

$$(\tilde{x}, \tilde{y}) = (s\theta, s\phi) + (\tilde{x}_c, \tilde{y}_c)$$

$s = f$ gives less distortion



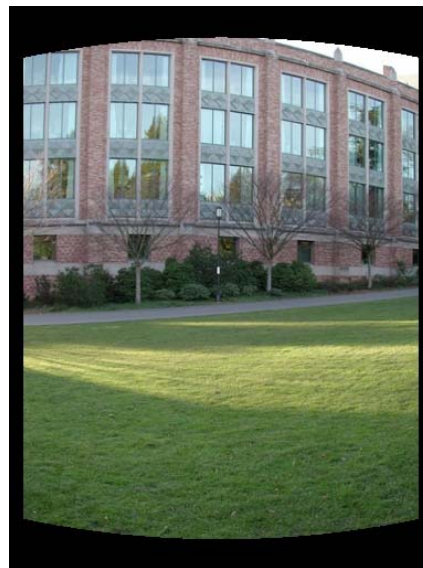
Spherical reprojection



input



$f = 200$ (pixels)



$f = 400$



$f = 800$

- Map image to spherical coordinates
 - need to know the focal length

Aligning spherical images



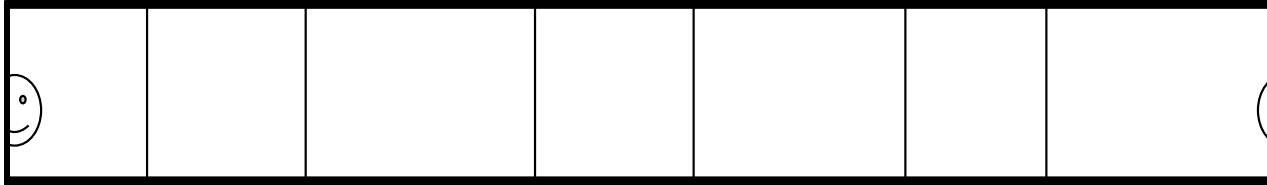
- Suppose we rotate the camera by θ about the vertical axis
 - How does this change the spherical image?

Aligning spherical images



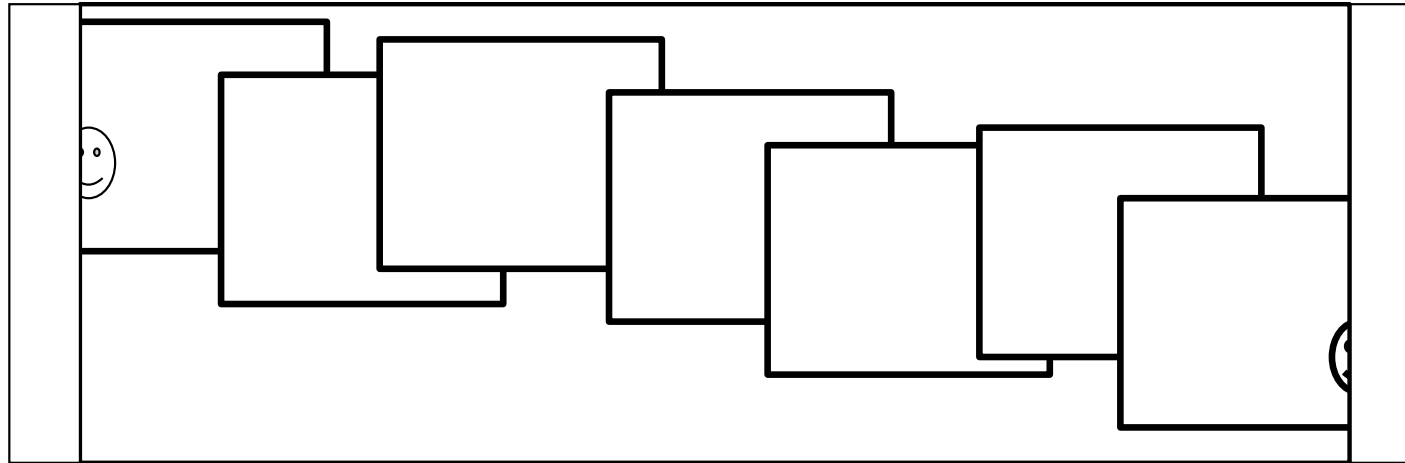
- Suppose we rotate the camera by θ about the vertical axis
 - How does this change the spherical image?
 - Translation by θ
 - This means that we can align spherical images by translation

Assembling the panorama



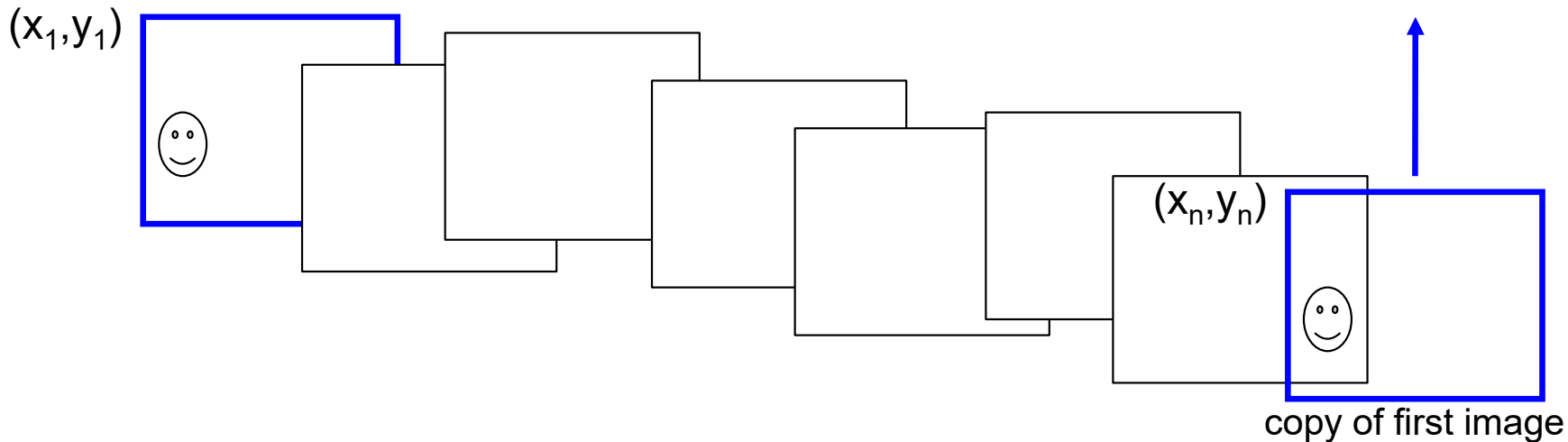
- Stitch pairs together, blend, then crop

Problem: Drift



- Error accumulation
 - small errors accumulate over time

Problem: Drift



- **Solution**

- add another copy of first image at the end
- this gives a constraint: $y_n = y_1$
- there are a bunch of ways to solve this problem
 - add displacement of $(y_1 - y_n)/(n - 1)$ to each image after the first
 - **apply an affine warp: $y' = y + ax$**
 - run a big optimization problem, incorporating this constraint
 - best solution, but more complicated
 - known as “bundle adjustment”



Unwrapping a sphere

Credit: JHT's Planetary Pixel Emporium



Spherical panoramas



Microsoft Lobby: <http://www.acm.org/pubs/citations/proceedings/graph/258734/p251-szeliski>

Different projections are possible



Blending

- We've aligned the images – now what?

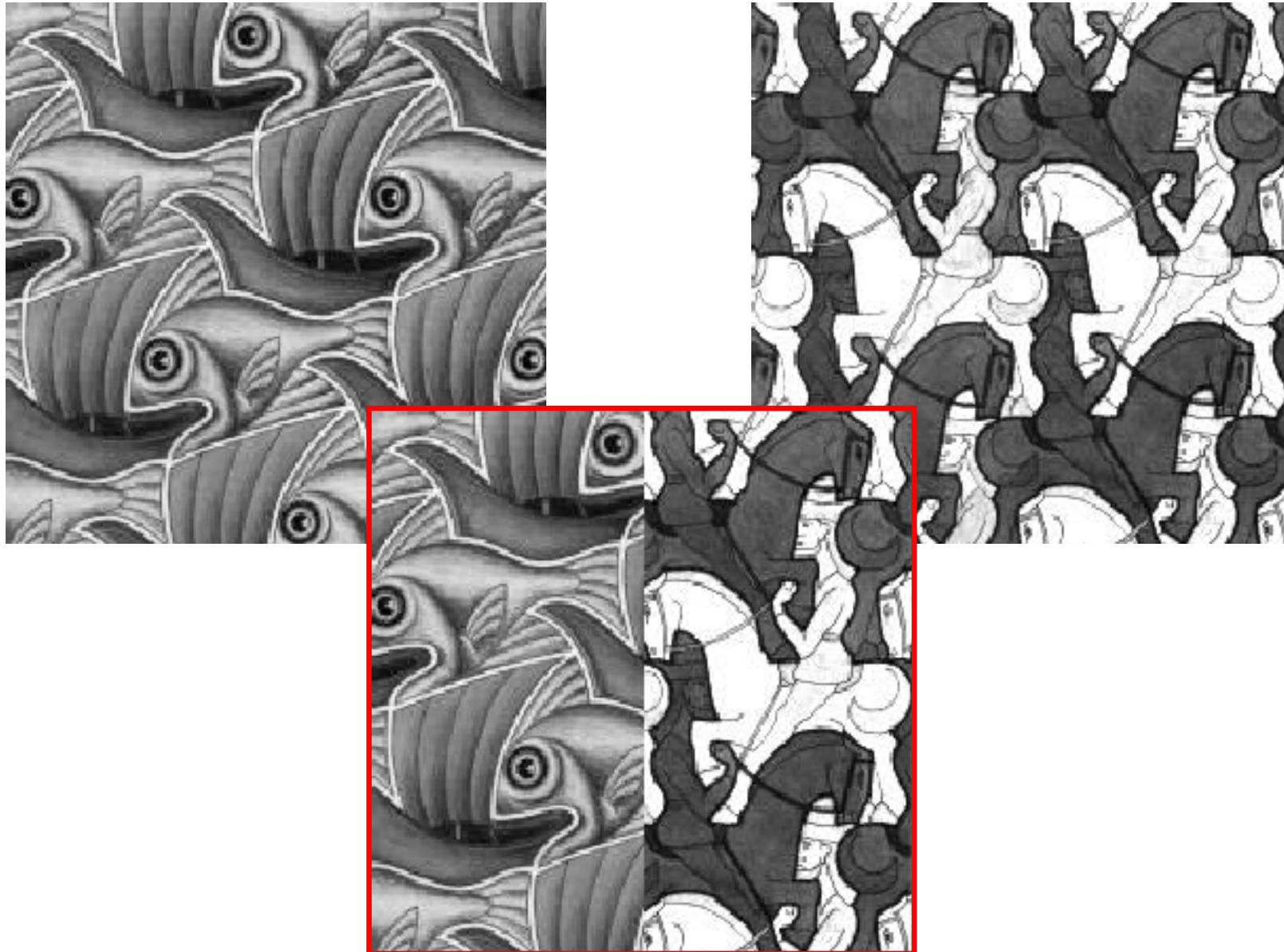


Blending

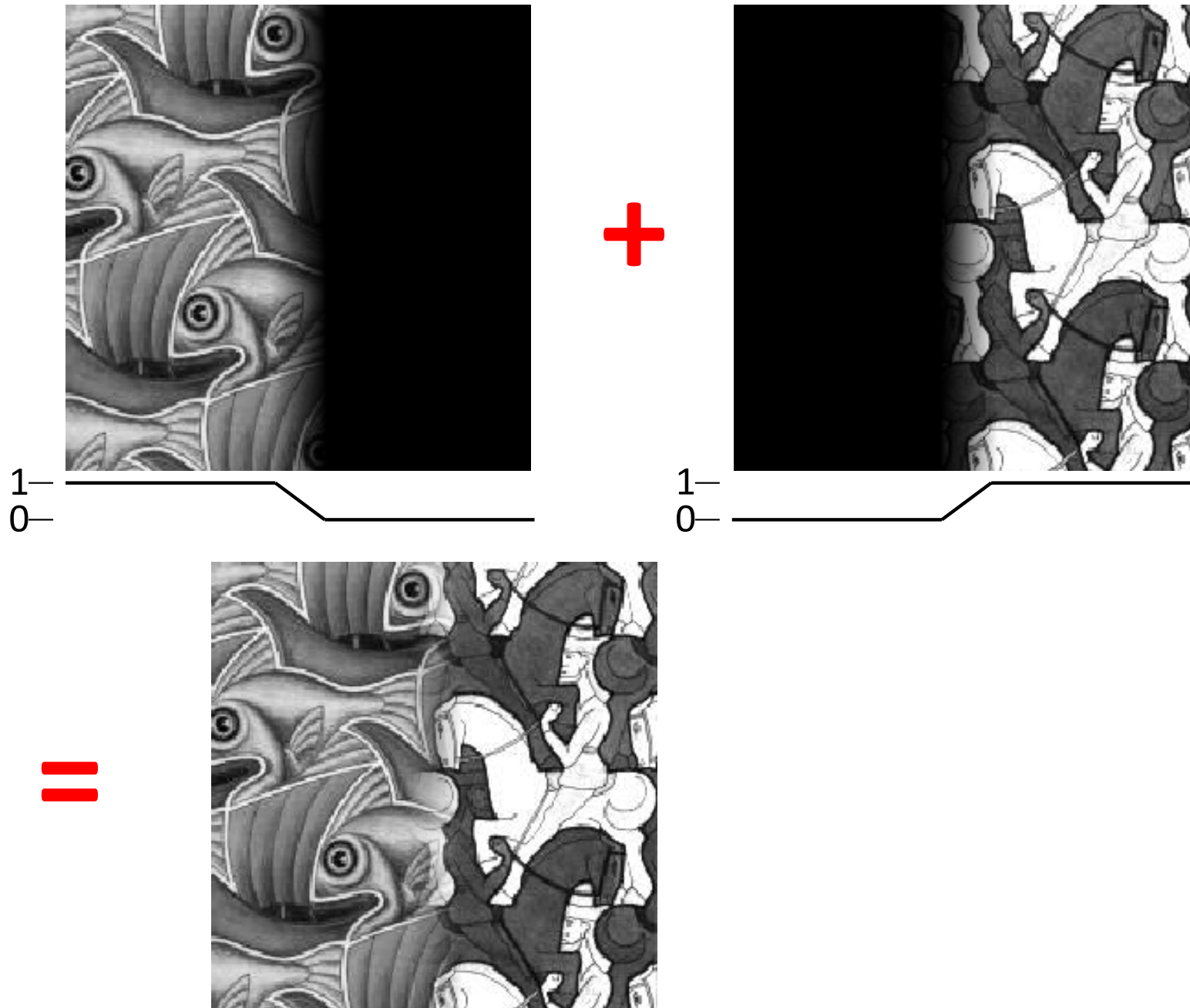
- Want to seamlessly blend them together



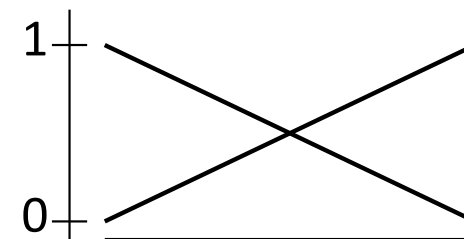
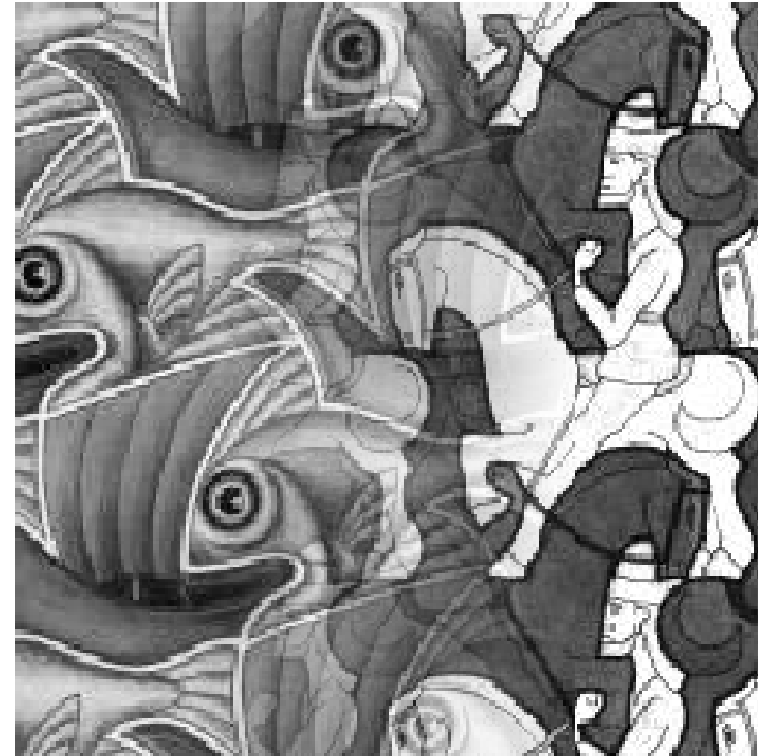
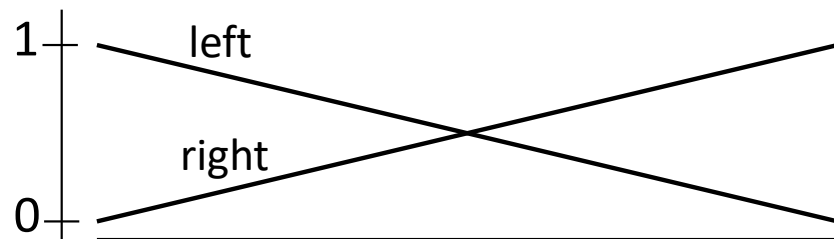
Image Blending



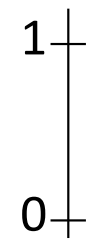
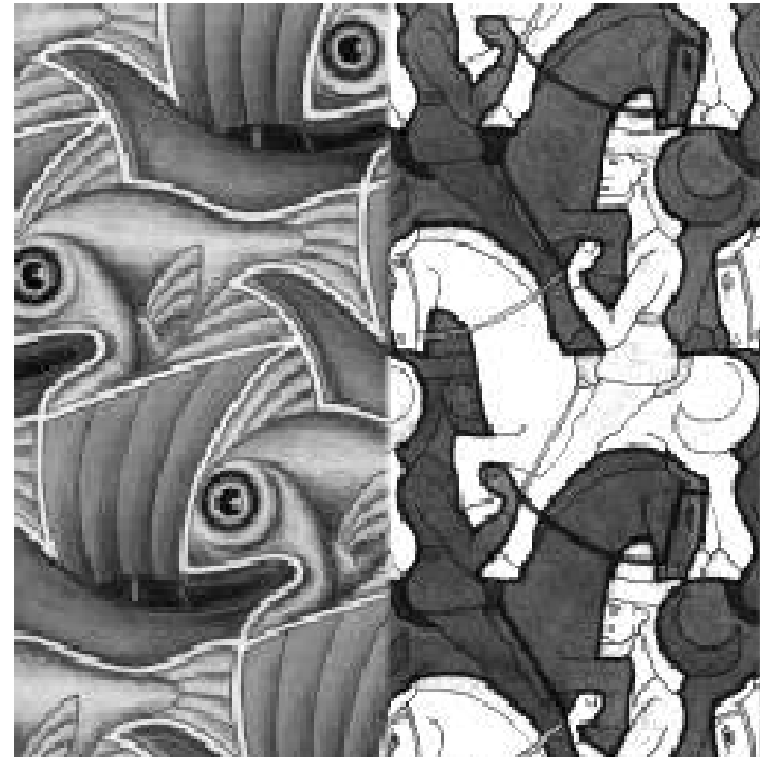
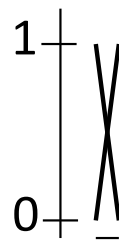
Feathering



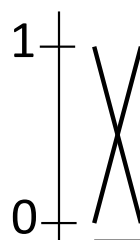
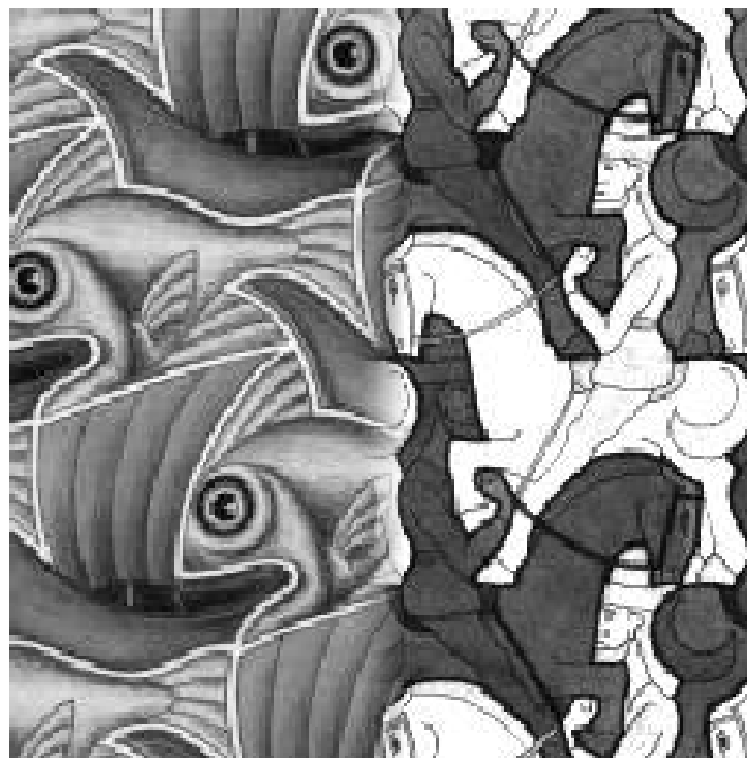
Effect of window size



Effect of window size



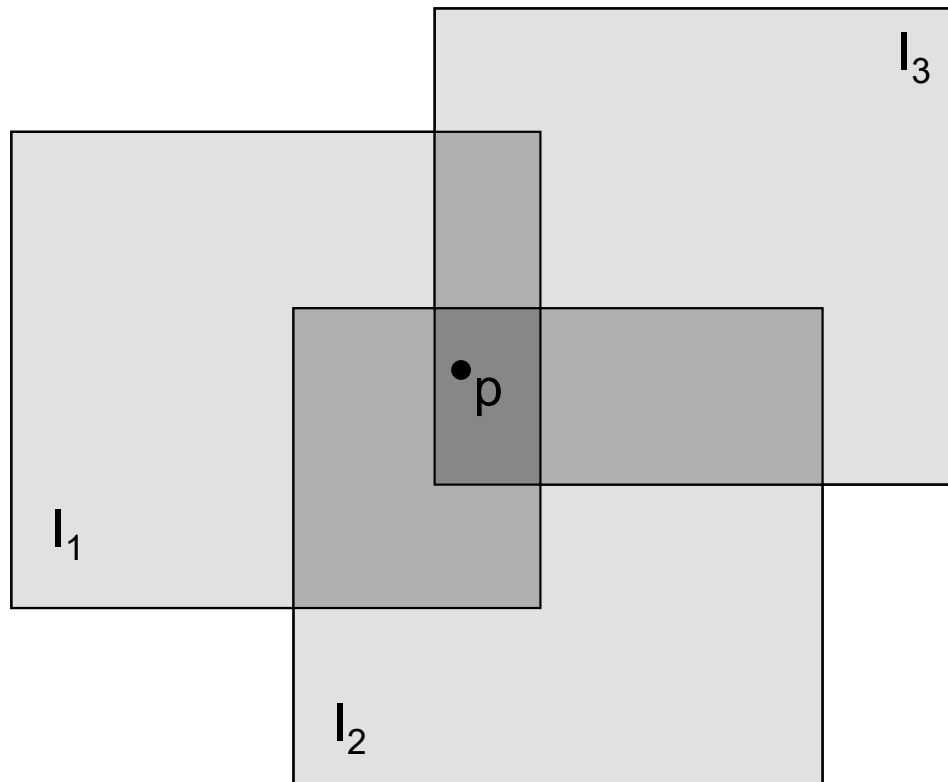
Good window size



“Optimal” window: smooth but not ghosted

- Doesn't always work...

Alpha Blending



Optional: see Blinn (CGA, 1994) for details:

<http://ieeexplore.ieee.org/iel1/38/7531/00310740.pdf?isNumber=7531&prod=JNL&arnumber=310740&arSt=83&ared=87&arAuthor=Blinn%2C+J.F.>

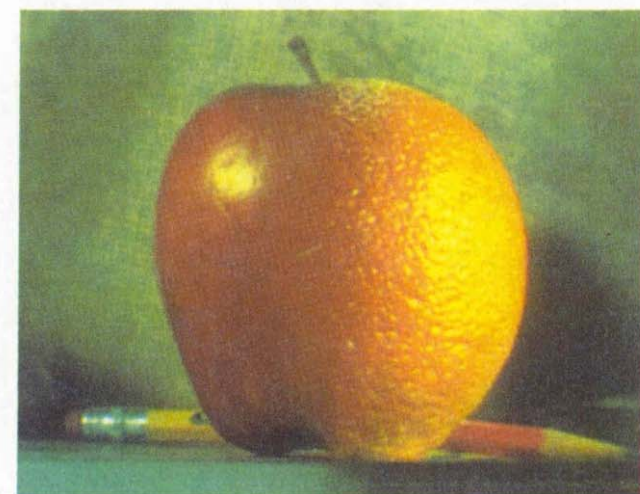
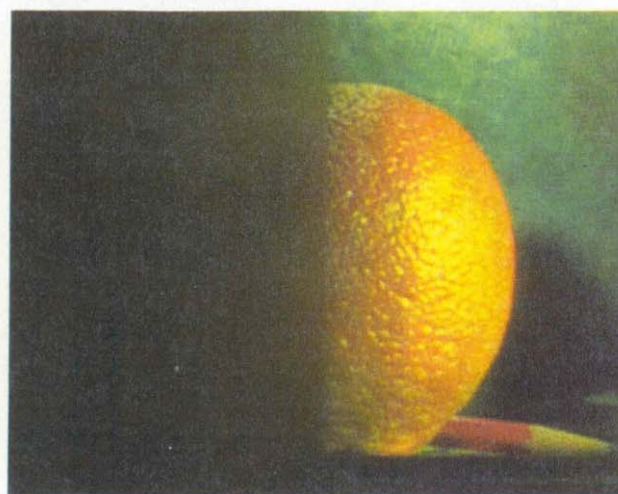
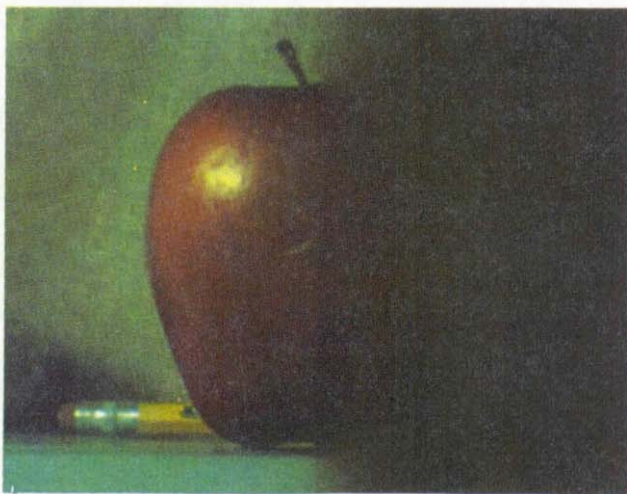
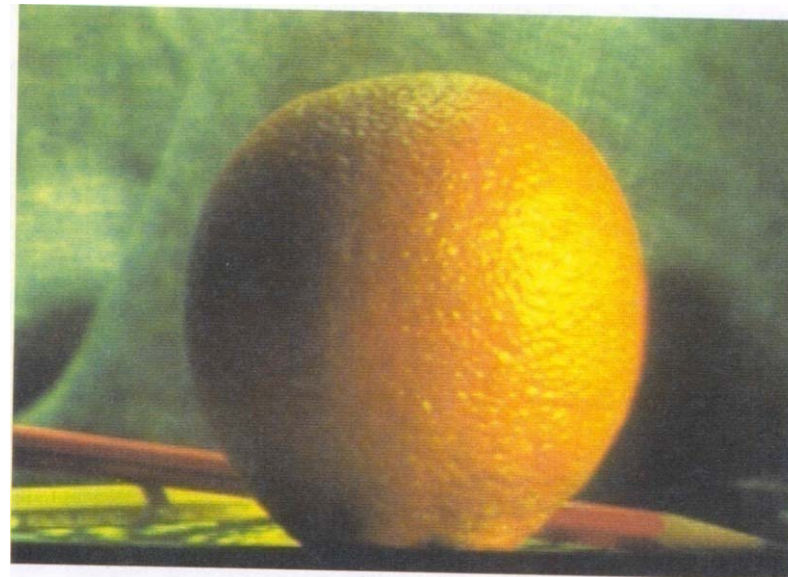
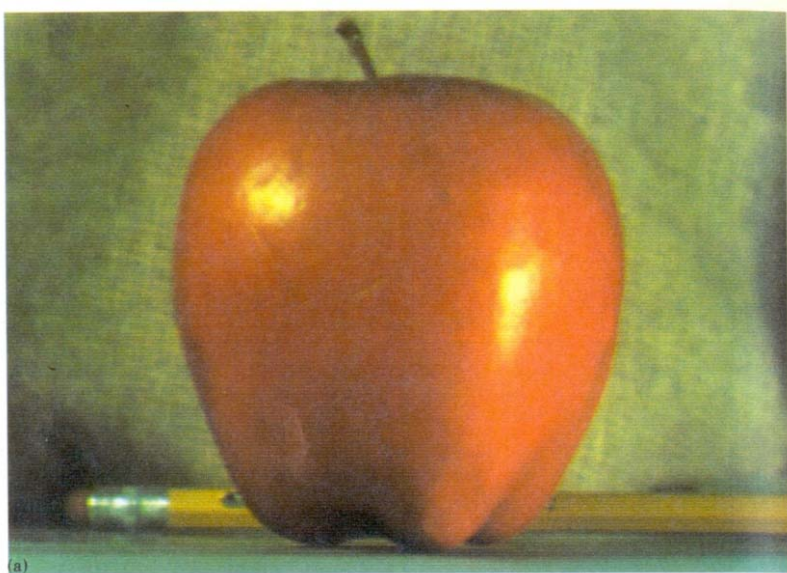
Encoding blend weights: $I(x,y) = (\alpha R, \alpha G, \alpha B, \alpha)$

color at p =
$$\frac{(\alpha_1 R_1, \alpha_1 G_1, \alpha_1 B_1) + (\alpha_2 R_2, \alpha_2 G_2, \alpha_2 B_2) + (\alpha_3 R_3, \alpha_3 G_3, \alpha_3 B_3)}{\alpha_1 + \alpha_2 + \alpha_3}$$

Implement this in two steps:

1. accumulate: add up the (α premultiplied) RGB α values at each pixel
2. normalize: divide each pixel's accumulated RGB by its α value

Pyramid blending



Create a Laplacian pyramid, blend each level

- Burt, P. J. and Adelson, E. H., [A multiresolution spline with applications to image mosaics](#), ACM Transactions on

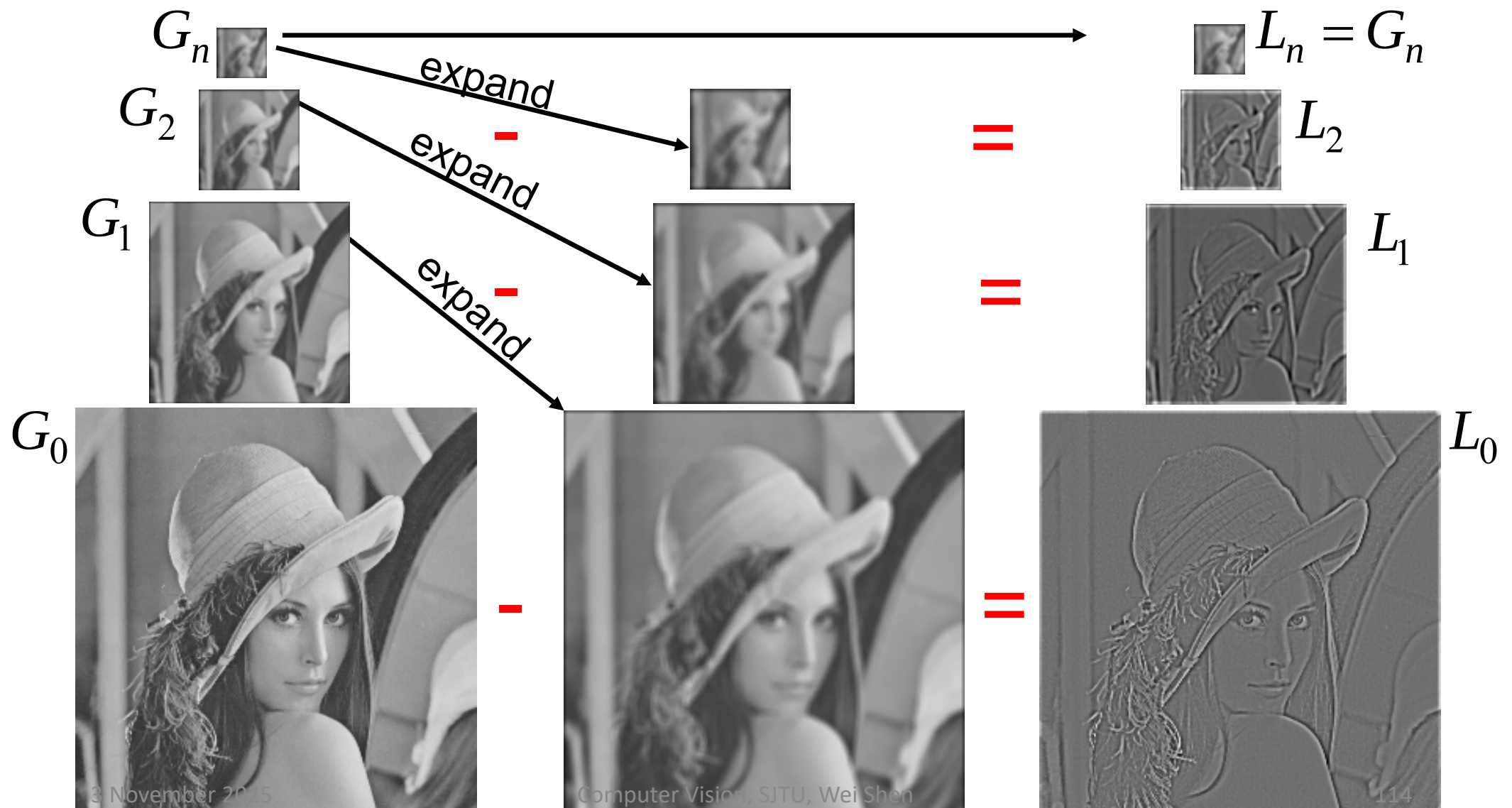
The Laplacian Pyramid

$$L_i = G_i - \text{expand}(G_{i+1})$$

Gaussian Pyramid

$$G_i = L_i + \text{expand}(G_{i+1})$$

Laplacian Pyramid



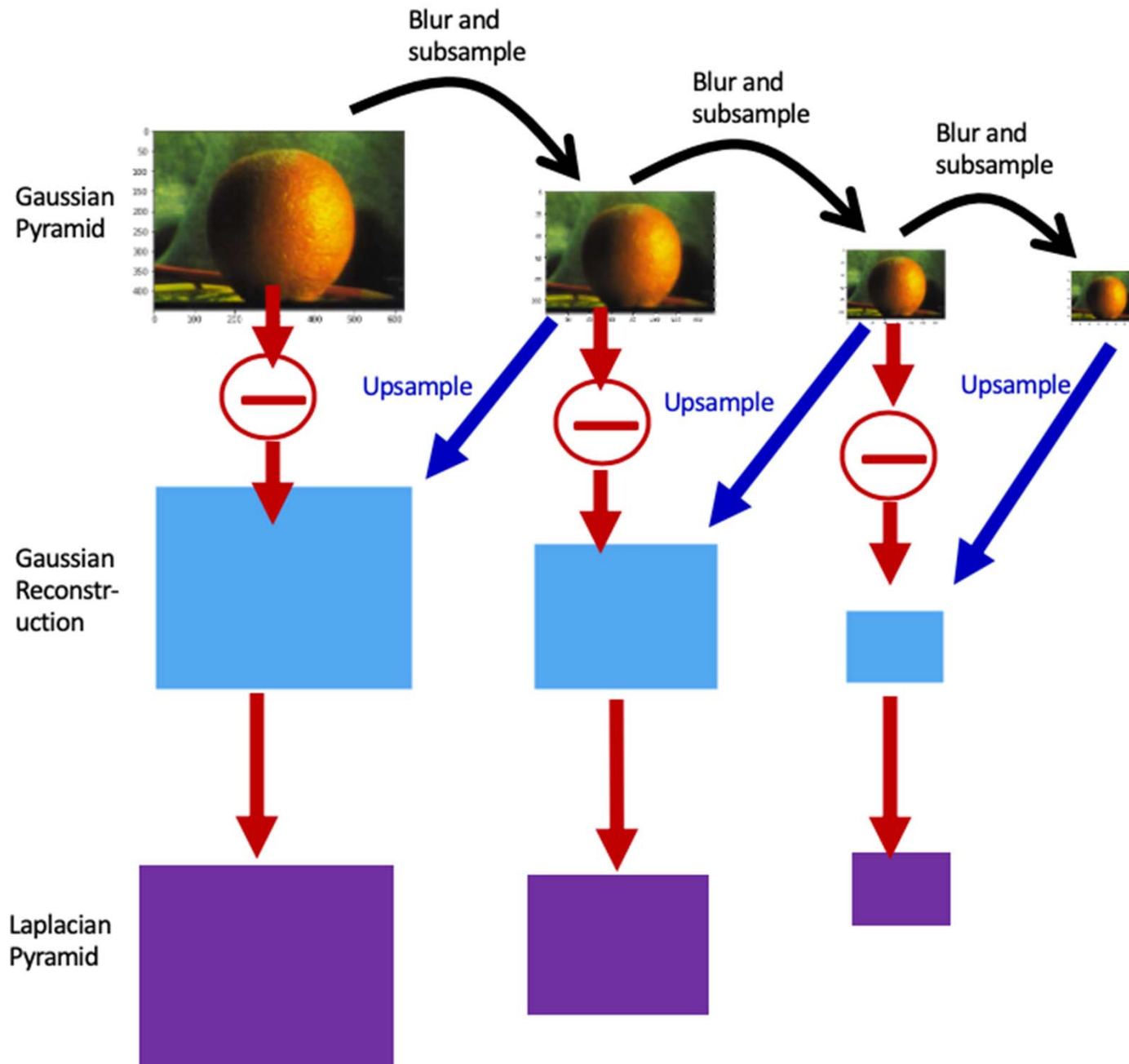
The Laplacian Pyramid



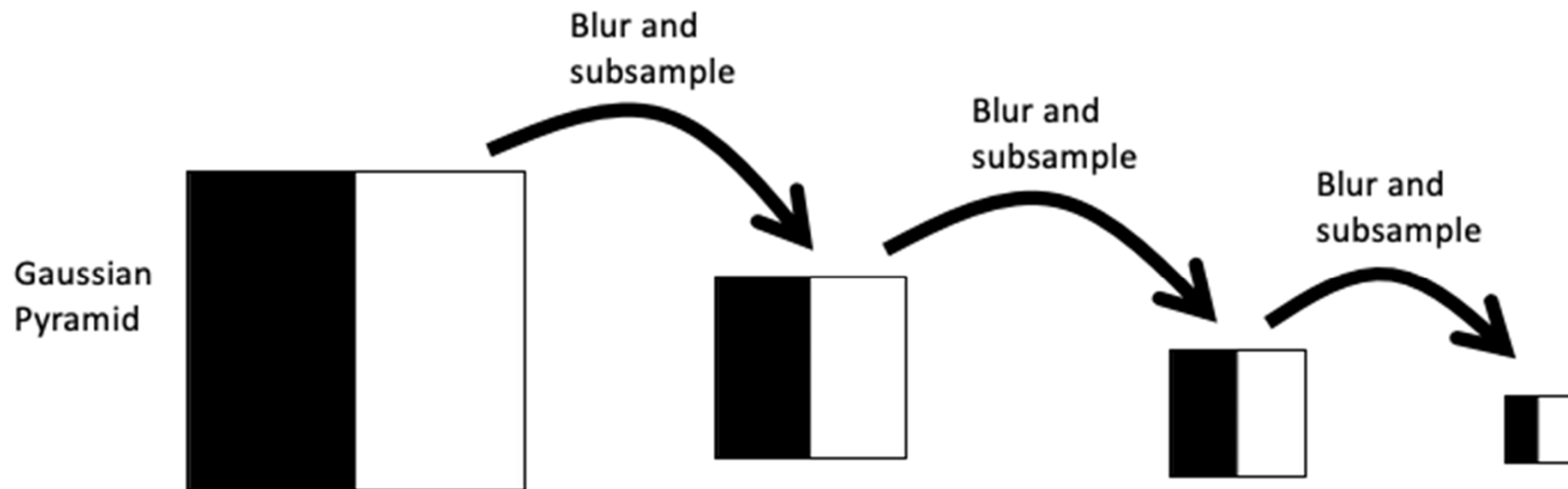
The Laplacian Pyramid



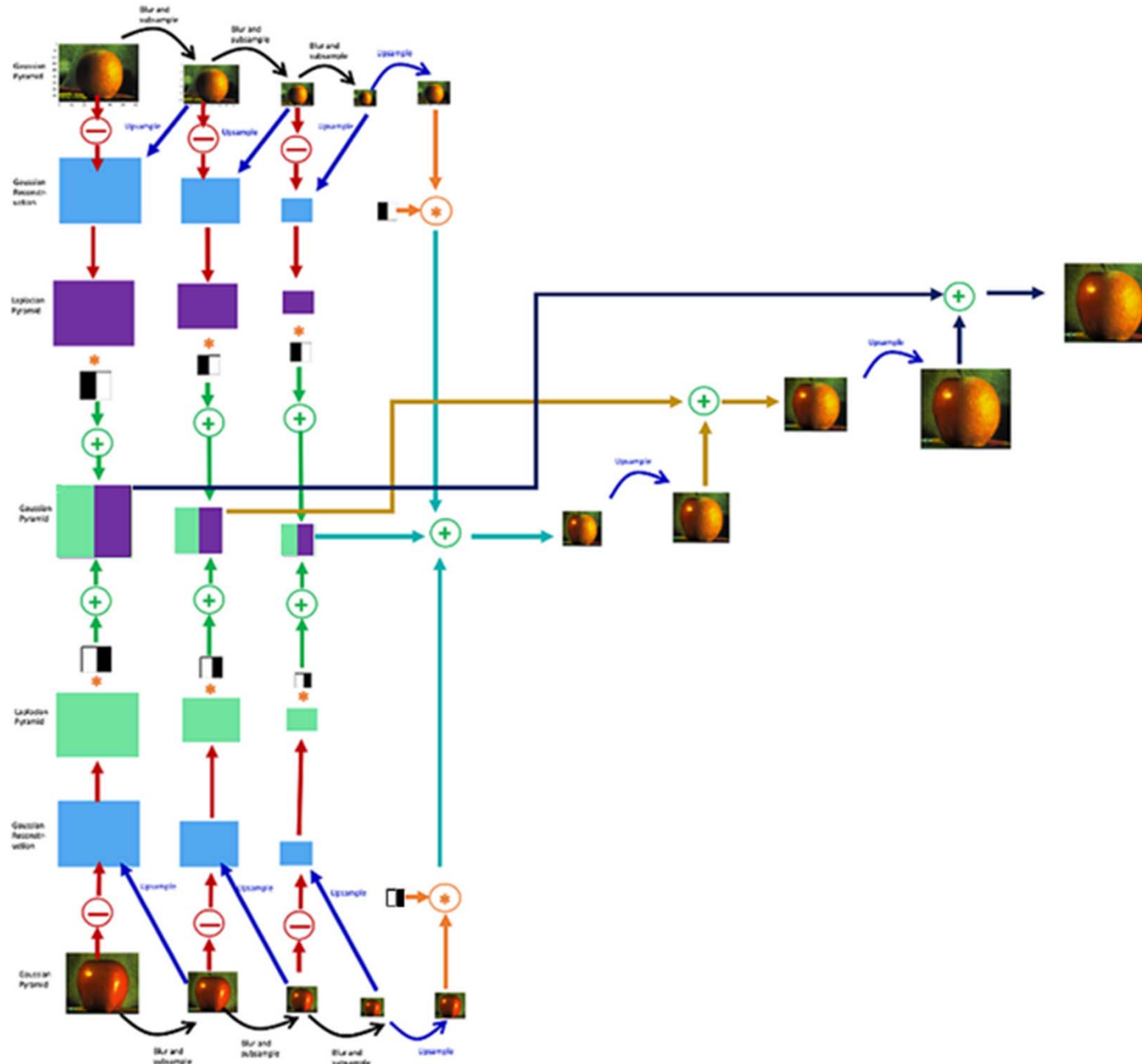
The Laplacian Pyramid



The Laplacian Pyramid



Pyramid blending



Poisson Image Editing



sources/destinations



cloning



seamless cloning

- For more info: Perez et al, SIGGRAPH 2003

Poisson Image Editing

Drag-and-Drop Pasting

SIGGRAPH 2006

Submission ID# 247

- For more info: Perez et al, SIGGRAPH 2003

Magic: ghost removal



M. Uyttendaele, A. Eden, and R. Szeliski.

Eliminating ghosting and exposure artifacts in image mosaics.

In Proceedings of the International Conference on Computer Vision and Pattern Recognition, volume 2, pages 509--516, Kauai, Hawaii, December 2001.

Magic: ghost removal



M. Uyttendaele, A. Eden, and R. Szeliski.

Eliminating ghosting and exposure artifacts in image mosaics.

In Proceedings of the International Conference on Computer Vision and Pattern Recognition, volume 2, pages 509--516, Kauai, Hawaii, December 2001.